Bases de Données Réparties

Ladjel Bellatreche

bellatreche@ensma.fr
05 49 49 80 72

Références bibliographiques

- Principles of Distributed Database Systems, Second Edition- M. Tamer Özsu & Patrick Valduriez
- http://www.cs.ualberta.ca/~database/ddbook.html
- Distributed Databases : principles & systems- Stefano Ceri and Giuseppe Pelagatti

Data Trucs

- Distributed database
- Data Warehouse (entrepôt de données)
- Data Marts (entrepôt fragmenté)
- WebData Warehouse
- Data Mining (fouille des données)
- Data Integration
- Data Engineering

Motivation

Relation employé E (#,nom,loc,sal,...)

40% des requêtes:

Q_a: SELECT *
FROM E
WHERE loc=S_a
AND...

40% des requêtes :

Q_b: SELECT *
FROM E
WHERE loc=S_b
AND ...

Deux sites : S_a, S_b

$Q_a \rightarrow$ Sa

Sb $\leftarrow Q_b$

Bases de données réparties (BDR)

- Différents niveaux de répartition
  - Données
  - Schémas ou catalogues de la BD
  - SGBD
  - Traitement (requêtes, transactions)
  - Composants matériels: mémoires, disques, ...

BDR = BD + Réseau

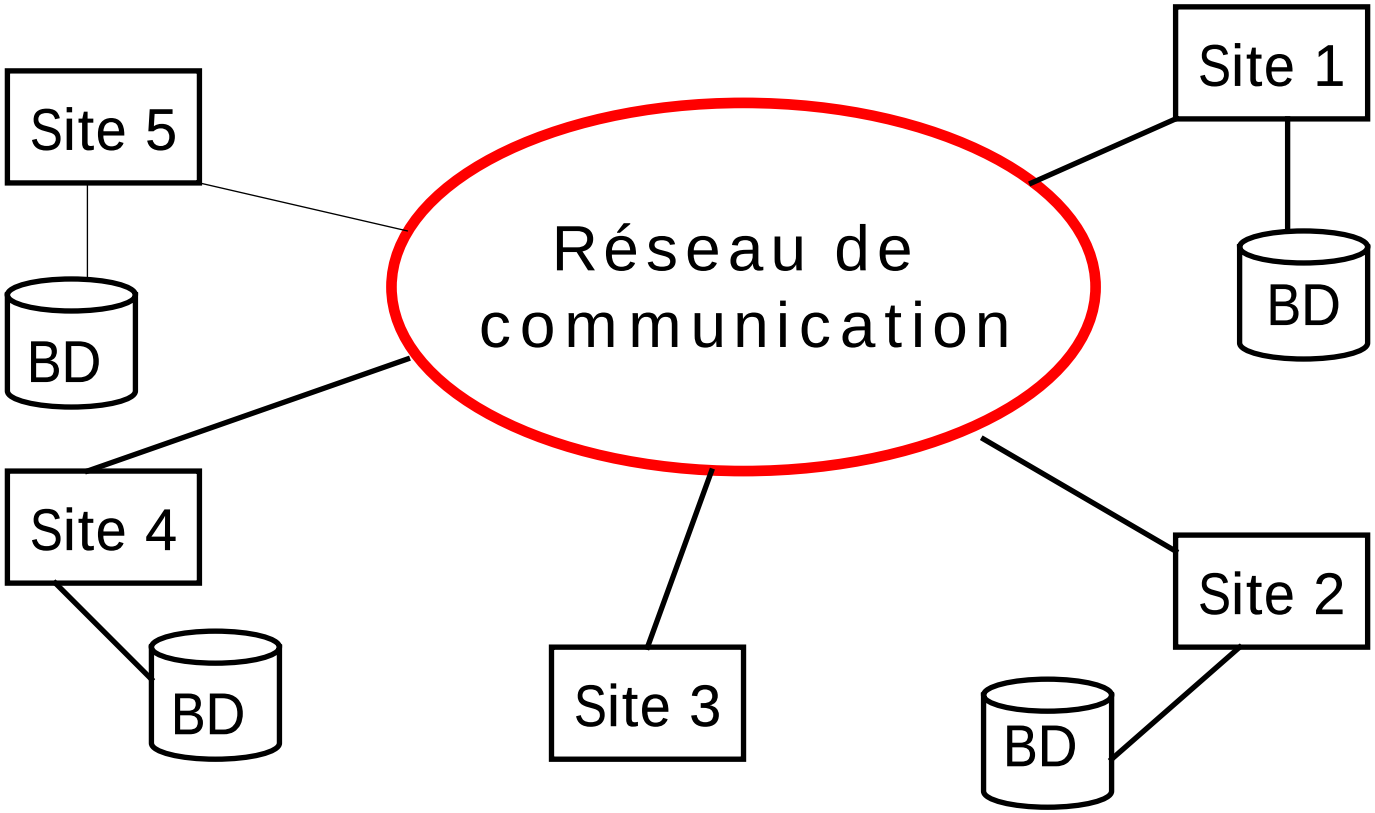
- BD répartie (distributed database)
  - Ensemble de BDs gérées par des sites différents et qui apparaissent à l'utilisateur comme une base unique
- SGBD Réparti (SGBDR)
  - Logiciel qui gère une BDR et qui rend la répartition transparente
- Client de SGBDR
  - Application qui accède aux informations distribuées par les interfaces du SGBDR

Applications

- Cas de grosses entreprises ou organismes ayant des agences géographiquement distribuées:
  - Banques
  - Fabrication
  - Médicales (BD biologiques)
  - Militaires
  - Systèmes de réservation de compagnies aériennes
  - WEB

Environnement

Sites = machines



Objectifs des BDRs

- Autonomie locale
- Transparence
- Performance améliorée
- Fiabilité et disponibilité accrues
- Partage accru de données et ressources
- Expansion graduelle

Nouveaux défis (I)

- Conception d'une BDR
  - Fragmentation
  - Allocation
  - Réplication (totale ou partielle)
- Transparence à la répartition
  - Extension de la notion d'indépendance logique et physique des données
  - Localisation (réplication, fragmentation)
    - Aucune spécification de la localisation des données

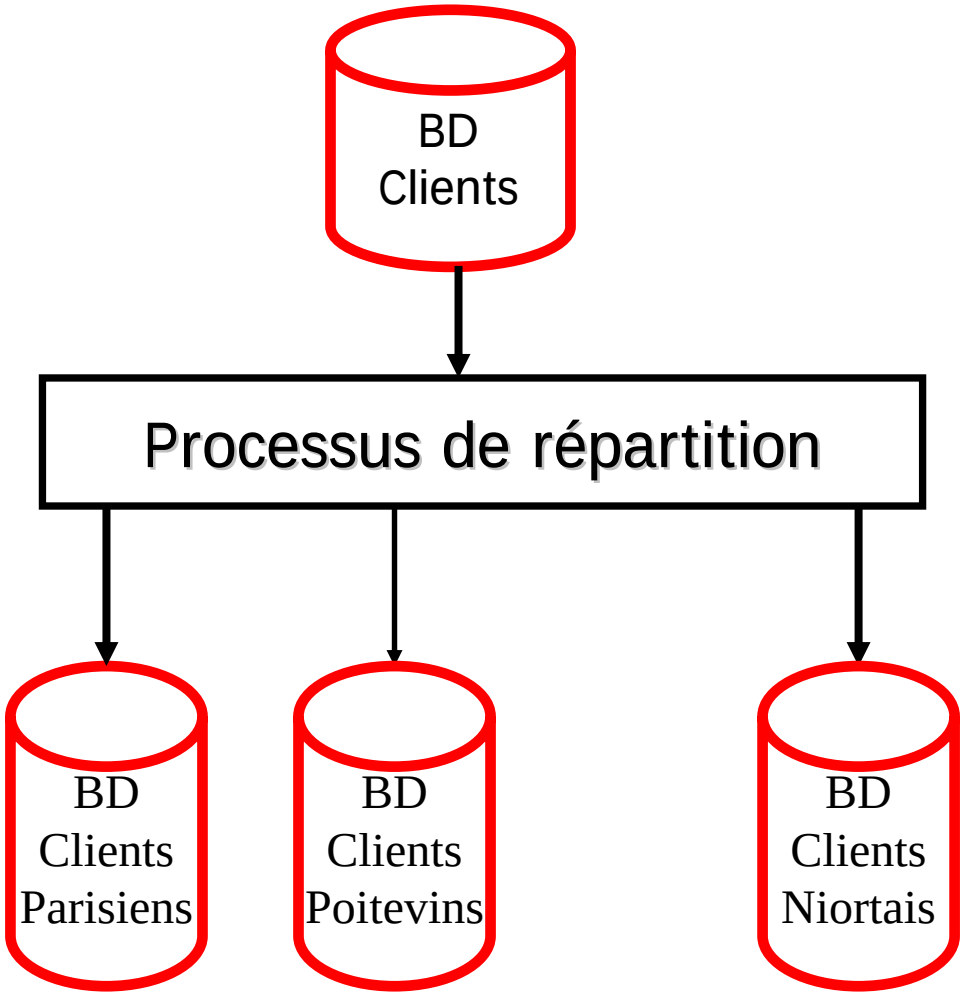
Nouveaux défis(II)

- Optimisation de requêtes réparties
  - Choix de la copie en lecture
  - Mise à jour de toutes les copies
  - Plan d'exécution réparti
- Transactions réparties
  - Maintien des propriétés ACID des transactions
  - Utilisateur aura à formuler ses transactions de la même manière que dans un environnement centralisé

Types de BDR

- **BDR homogène**
  - Obtenue en divisant une BD en un ensemble de BD locales, chacune étant gérée par le même SGBD
  - Même modèle de données
  - Même langage de requêtes
    - Exemple: DB2, ORACLE (SQL)
  - Données de la base sont réparties sur plusieurs sites

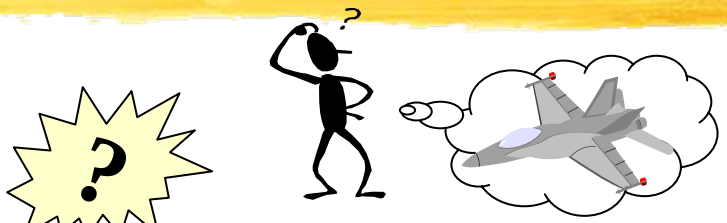
Exemple


BD
Clients
Processus de répartition
BD
Clients
Parisiens
BD
Clients
Poitevins
BD
Clients
Niortais

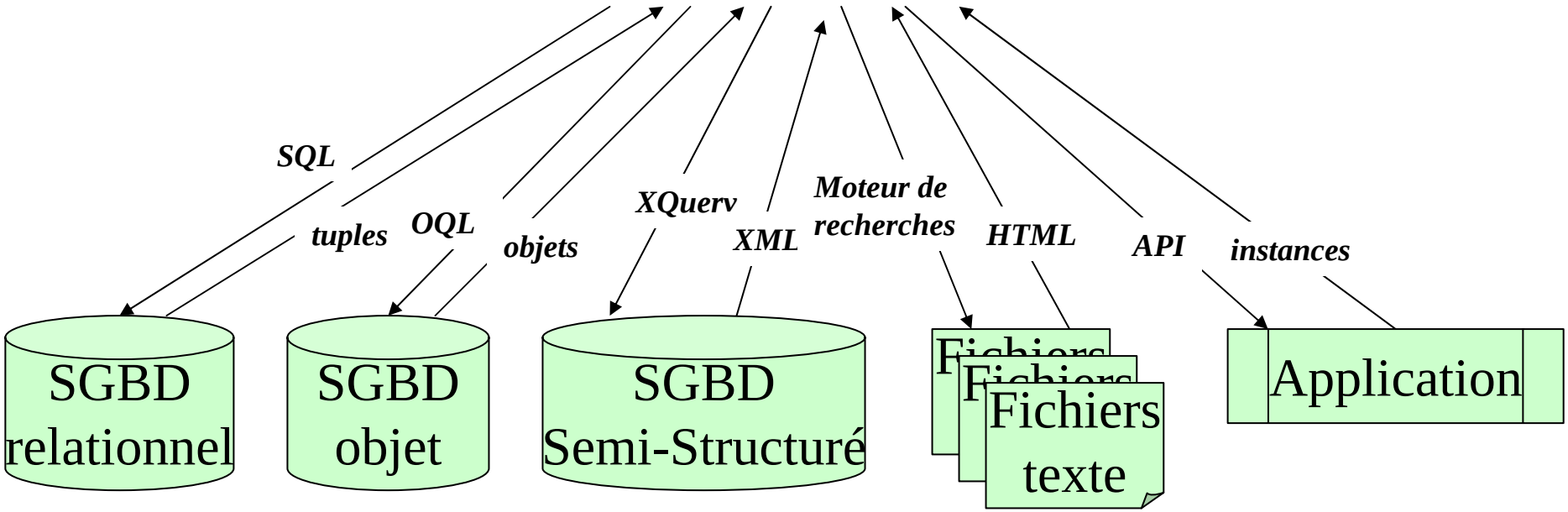
BDR Hétérogènes

- Deux niveaux d'hétérogénéité:
  - Les BD ont le même modèle (relationnel) mais sont gérées par des SGBD différents (Oracle, SQL server, ...)
  - Les BD ont des modèles différents (relationnel, objet) et gérées par des SGBD différents (Oracle, O2)
- BDR hétérogène
  - BD répartie obtenue en **intégrant** dans une BD unique un ensemble de BD locales gérées par des SGBD différents

Exemple



Exemple : Chercher où passer les vacances cet été.



Agence de voyage | *Chaîne hotelière* | *Site horaire des vols* | *Informations Pays* | *Météo*

Définition d'une BDR

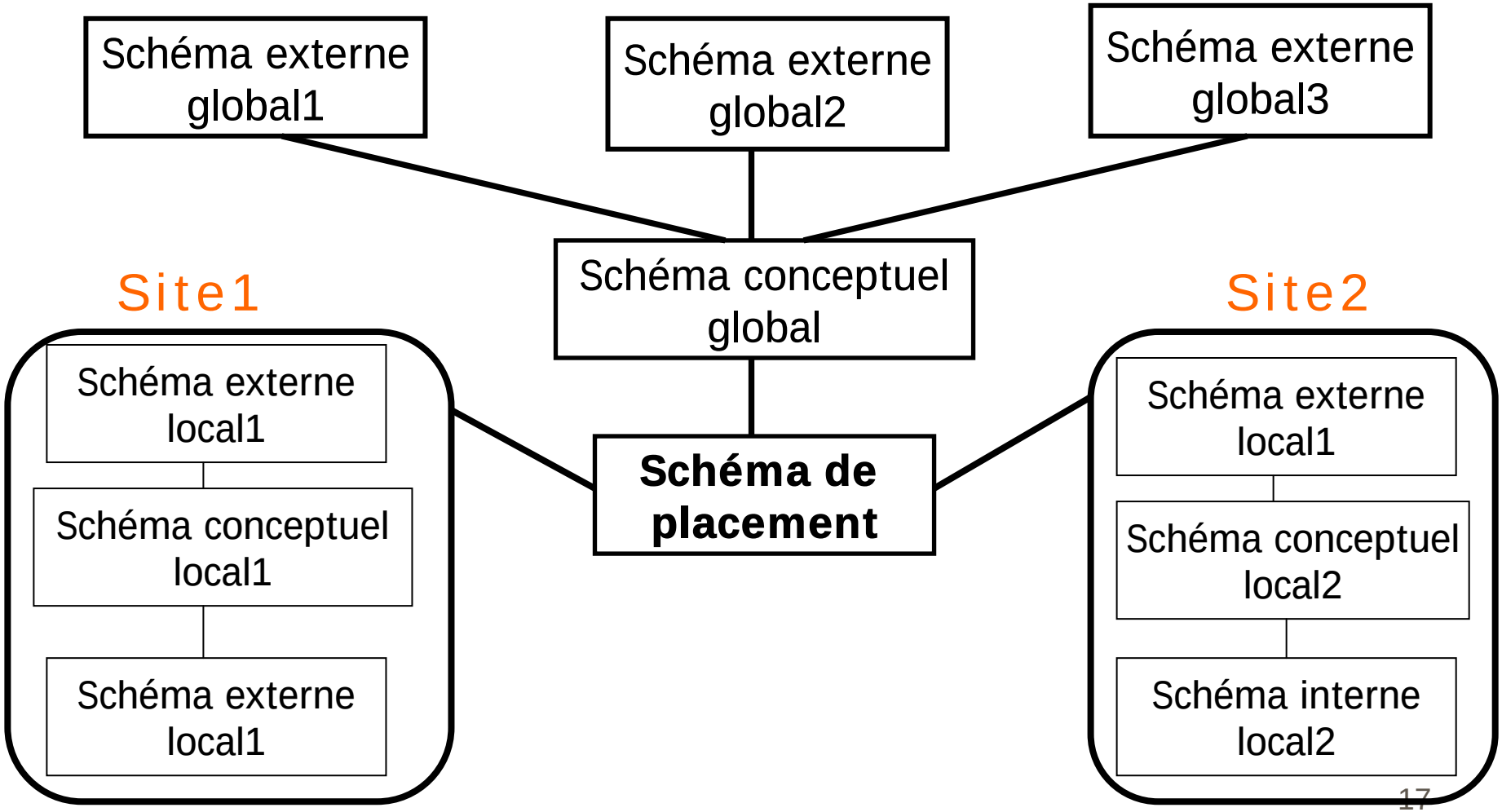
- Site local
- Site de naissance (ne change pas)
- Site de stockage (peut changer)
- Site de l'usager
- Dupont de site S1 crée une relation R et la stocke dans S2

CREATE TABLE WAGON(NW, Type, Poids, Gare, Etat) ON S2

- S1 = site de naissance (WAGON @ S1)
- S2 = site de stockage
- Dupont de S1 déplace la relation de S2 vers S3

MIGRATE TABLE WAGON@S1 TO S3

Architecture des schémas d'une BDR

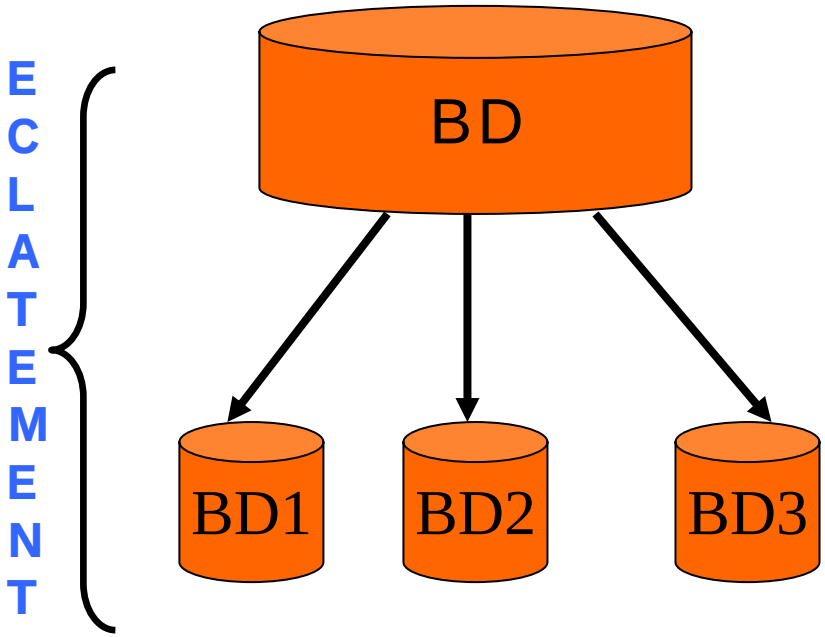


Conception des BDR

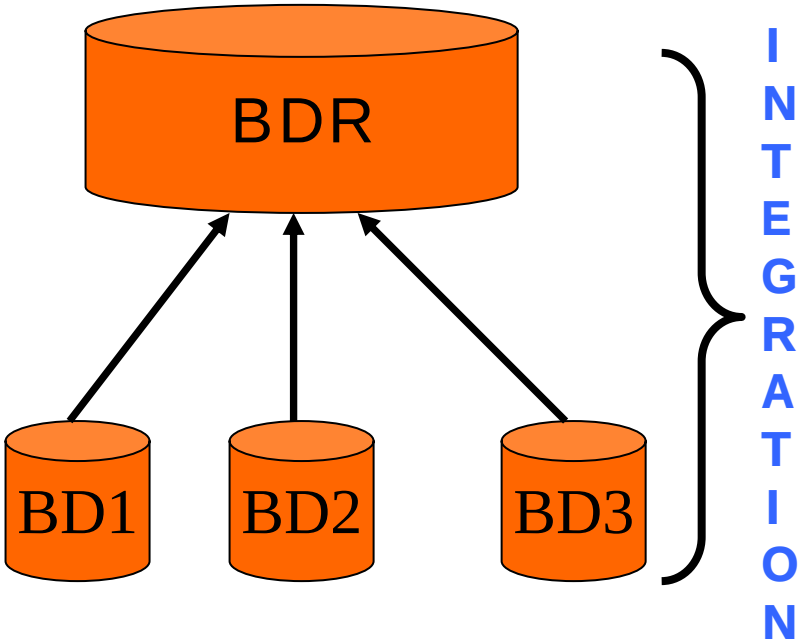
- **Approche descendante**
  - Environnement homogène
  - Conception à partir de **zéro**
  - Nouvelles étapes avant la conception physique
    - Localisation des données
    - Schémas locaux
- **Approche ascendante**
  - On part de BD existantes (souvent hétérogènes)

Conception des BDR

Conception descendante



Conception ascendante



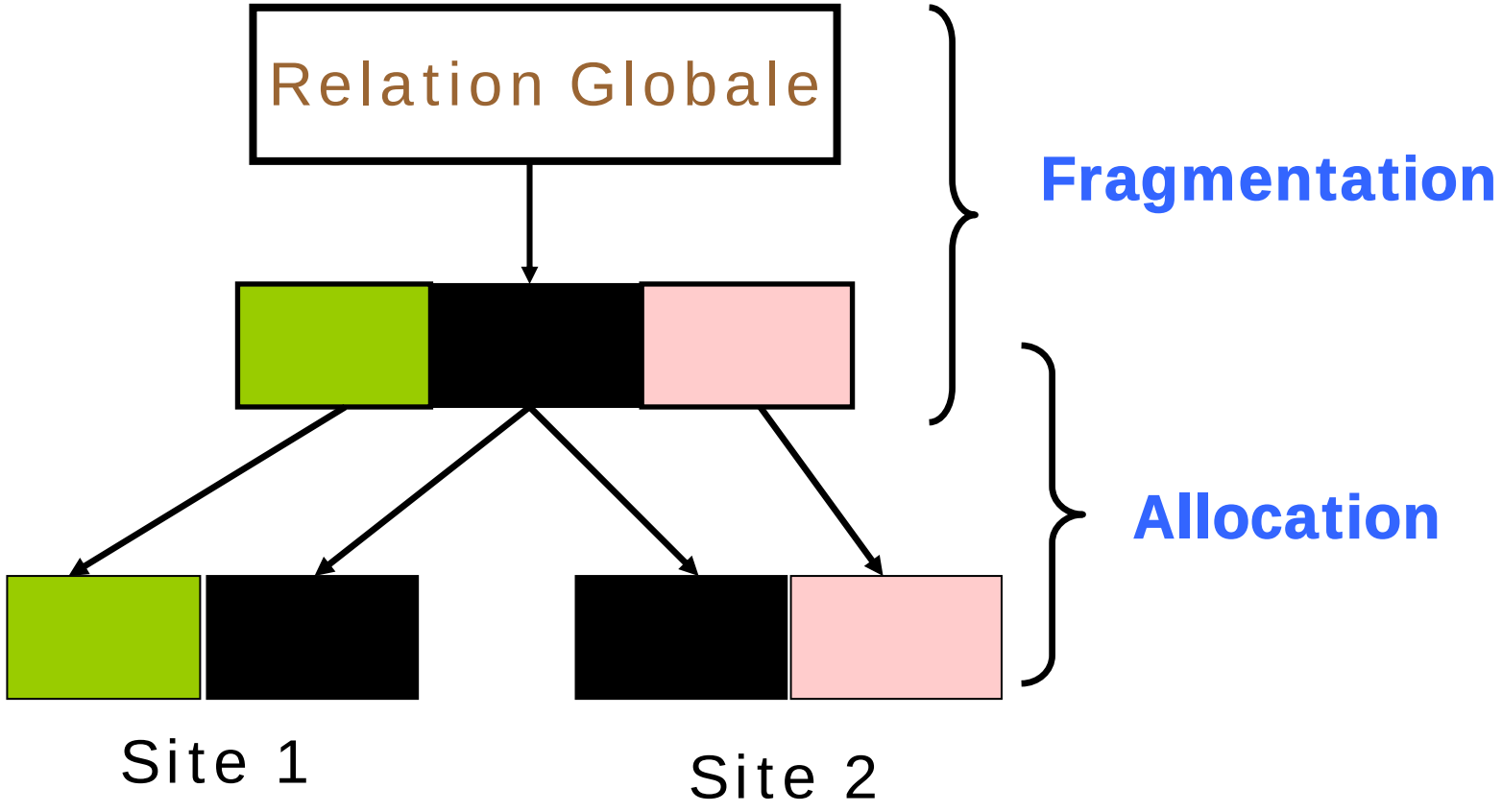
Approche ascendante

- Intégration des BD locales existantes dans une seule base
- La distribution des données est préexistante
- Sémantique des schémas participants

Approche descendante

- La distribution des données est bien présente
- Les tables du schéma global sont fragmentées (processus de fragmentation)
- Fragment
  - Sous-table obtenue par sélection de lignes et de colonnes à partir d'une table globale
- Les fragments sont donc placés sur des sites (processus d'allocation)

Objectifs de la décomposition



Techniques de fragmentation

- Problème de fragmentation:
  - Entrée: une relation d'un schéma global
  - Sortie: un schéma de fragmentation (ensemble de fragments)
- Différents types de fragmentation
  - Horizontale
  - Verticale
  - Mixte

Fragmentation horizontale

Décomposition de la table en groupes de lignes

Table Client(N°Client, Nom, Ville)

➔ Deux types de fragmentation horizontale:

- Primaire
- Dérivée

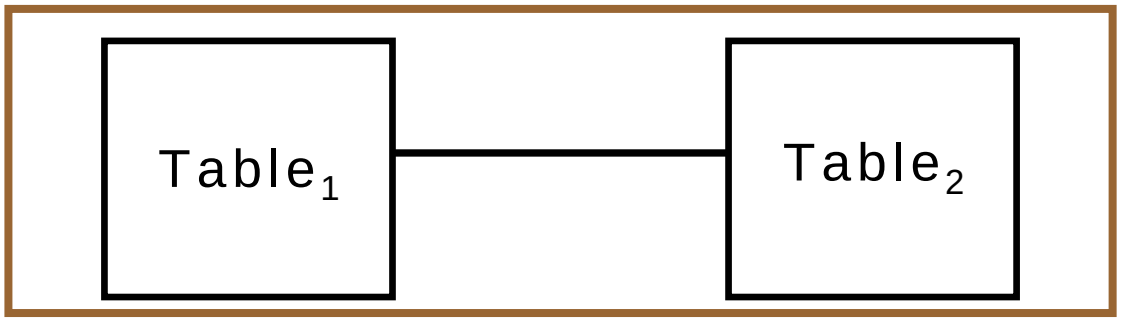
Fragmentation horizontale primaire

- Obtention des fragments horizontaux:
  - Fragmentation horizontale est définie par l'opération de sélection
- Exemple
  - Client(N°Client, Nom, Ville) peut être fragmentée :
  - Client1= SELECT * FROM Client WHERE Ville = "Paris"
  - Client2= SELECT * FROM Client WHERE Ville <> "Paris"
- Reconstruction de la relation initiale:

  Client = Client1 ∪ Client2

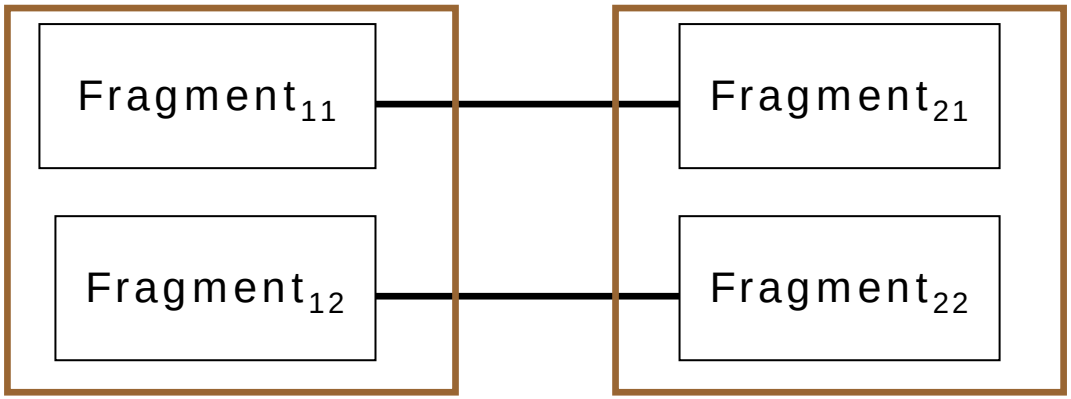
Fragmentation horizontale dérivée(I)

➔ Fragmentation d'une table en fonction des fragments horizontaux d'une autre table



Avant fragmentation

Fragments dérivés



Après fragmentation

Fragmentation horizontale dérivée(II)

- Obtention des fragments horizontaux dérivés:
  - Fragments dérivés sont obtenus par l'opération de semi-jointure
- Exemple
  - Commande(N°Client, N°Produit, Date, Qte, N°Représentant)
  - Commande1= SELECT * FROM Commande
    WHERE N°client IN
    (SELECT N°Client FROM CLIENT1)
  - Commande2= SELECT * FROM Commande
    WHERE N°Client IN
    (SELECT N°Client FROM CLIENT2)
- Reconstruction de la relation initiale:

  Commande = Commande1 ∪ Commande2

Fragmentation verticale(I)

Décomposition de la table en groupes de colonnes

Table Client(N°Client, Nom, Sexe, Ville)

N°Client	Nom

Fragment vertical$_1$

N°Client	Sexe	Ville

Fragment vertical$_2$

Fragmentation verticale(II)

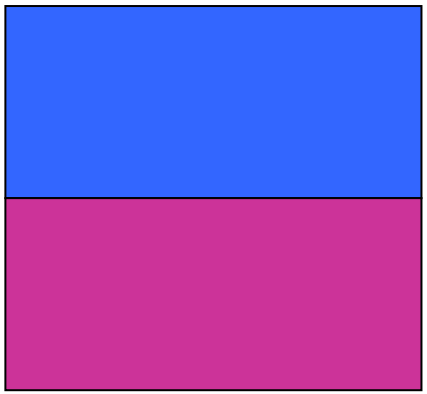
- Obtention des fragments verticaux:
  - Fragmentation verticale est définie par l'opération de projection
- Exemple
  - Client(N°Client, Nom, Sexe,Ville) peut être fragmentée :
  - $Client_1$= SELECT N°Client, Nom FROM Client
  - $Client_2$= SELECT N°Client, Sexe, Ville FROM Client
- Reconstruction de la relation initiale:

  Client = $Client_1$ join $Client_2$

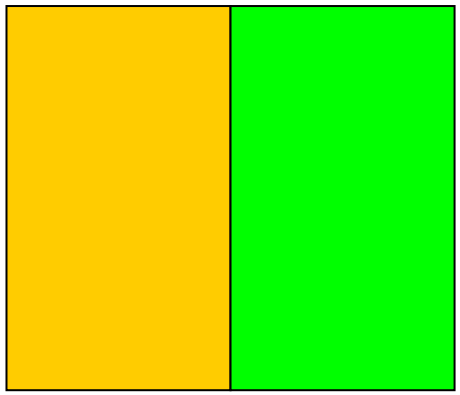
Pourquoi le N°Client est dupliqué dans les deux fragments?

Fragmentation mixte

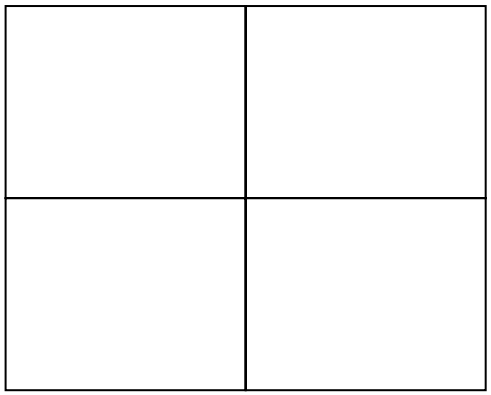
- Obtention des fragments mixtes:
  - Fragmentation mixte résulte de l'application successive d'opérations de fragmentation horizontale et de fragmentation verticale



Horizontale



Verticale



Mixte

Avantages et inconvénients de la fragmentation

+ Réduction des accès non pertinents
+ Parallélisme intra-requête
+ Combinée avec d'autres techniques d'optimisation (index, vues matérialisées, etc.)
– génération des fragments disjoints est un problème difficile
– Accès multiples aux fragments nécessitent des opérations de jointure et d'union
– La migration des données (conséquence d'une mauvaise fragmentation horizontale)

Règles de correction

- **Complétude**
  - Assure que tous les tuples d'une relation sont associés à au moins un fragment
- **Reconstruction**
  - Assure qu'une relation peut être reconstruite à partir de ses fragments
- **Disjonction**
  - Assure que les fragments d'une relation sont disjoints deux à deux

Règles de correction

- Tout algorithme de fragmentation doit satisfaire les règles correction
- Complétude
  - Verticale: Décomposition sans perte
  - Horizontale: ?
- Reconstruction
  - Verticale : opération de jointure
  - Horizontale : opération d'union

Comment Fragmenter?

- Fragmentation horizontale
  - Fragments horizontaux sont disjoints
- Deux catégories de méthodes de fragmentation
  - Data-driven partitioning (voir cours entrepôts)
    - Hash-partitioning (Oracle10g)
    - Range partitioning (Oracle10g)
    - List Partitioning (Oracle10g)
  - Query-Driven Partitioning
    - Prédicats définis dans les requêtes

Fragmentation dirigée par des requêtes

- Optimiser les requêtes les plus fréquentes
  - Règle 20/80;
  - Connaissance préalable des requêtes;
  - Fréquences d'accès des requêtes.
  - ➔ **Travail de l'administrateur de la BD**
- Changement de requêtes peut entraîner une re-fragmentation
  - ➔ Tuning de la base de données

Procédure de fragmentation (I)

- Informations sur la base de données
  - Prédicats simples :

    Étant donnée une relation $R(A_1, A_2, ..., A_n)$

    Un prédicat simple p_j défini sur R est défini:

    $$\mathbf{p_j : A_i\ \theta\ valeur}$$

    avec: $\theta \in \{=, <, >, \leq, \geq, \neq\}$; et valeur $\in$ domaine(D_i)

- Exemple

p1: Ville ="Poitiers"
p2: Salaire $\leq$ 10 000

Procédure de fragmentation(II)

- Soit $P_r = \{p_1, p_2, \ldots, p_m\}$ un ensemble de prédicats simples définis sur la relation R_i, l'ensemble de minterms $M = \{m_1, m_2, \ldots, m_z\}$ est défini comme suit:

$$\mathbf{M = \{m_i \mid m_i = \wedge_{Pj \in Pr}\ p^*_j\},}\ 1 \leq i \leq z,\ 1 \leq j \leq z;$$

$$\text{where } p^*_j = p_j \text{ or } \neg p_j$$

- Exemple:

m_1: (Ville ="Poitiers") $\wedge$ (Salaire $\leq$ 10 000)
m_2: NOT(Ville="Poitiers") $\wedge$ (Salaire $\leq$ 10 000)
m_3: (Ville ="Poitiers") $\wedge$ NOT(Salaire $\leq$ 10 000)
m_4: NOT(Ville ="Poitiers") $\wedge$ NOT(Salaire $\leq$ 10 000)

Informations sur la BD

- Sélectivité d'un minterm : $sel(m_i)$
  - Le nombre de tuples de la relation satisfaisant la clause du minterm.
- Fréquence d'accès d'une requête : $acc(q_i)$

Algorithme de génération des fragments horizontaux

- Génération des minterms
- Deuxième étape: élimination des minterms contradictoires
- Exemple
  - Considérons les prédicats simples suivants:

    A<10, A>5, Loc = “Poitiers”, Loc = “Paris”

$5 < A < 10$

Minterm predicates (I)

(1) $A<10 \wedge A>5 \wedge Loc=S_A \wedge Loc=S_B$

(2) $A<10 \wedge A>5 \wedge Loc=S_A \wedge \neg(Loc=S_B)$

(3) $A<10 \wedge A>5 \wedge \neg(Loc=S_A) \wedge Loc=S_B$

(4) $A<10 \wedge A>5 \wedge \neg(Loc=S_A) \wedge \neg(Loc=S_B)$

(5) $A<10 \wedge \neg(A>5) \wedge Loc=S_A \wedge Loc=S_B$

(6) $A<10 \wedge \neg(A>5) \wedge Loc=S_A \wedge \neg(Loc=S_B)$

(7) $A<10 \wedge \neg(A>5) \wedge \neg(Loc=S_A) \wedge Loc=S_B$

(8) $A<10 \wedge \neg(A>5) \wedge \neg(Loc=S_A) \wedge \neg(Loc=S_B)$

$A \leq 5$

Minterm predicates (II)

~~(9) $\neg(A<10) \wedge A>5 \wedge Loc=S_A \wedge Loc=S_B$~~

(10) $\neg(A<10) \wedge A>5 \wedge Loc=S_A \wedge \cancel{\neg(Loc=S_B)}$

(11) $\neg(A<10) \wedge A>5 \wedge \cancel{\neg(Loc=S_A)} \wedge Loc=S_B$

~~(12) $\neg(A<10) \wedge A>5 \wedge \neg(Loc=S_A) \wedge \neg(Loc=S_B)$~~

~~(13) $\neg(A<10) \wedge \neg(A>5) \wedge Loc=S_A \wedge Loc=S_B$~~

~~(14) $\neg(A<10) \wedge \neg(A>5) \wedge Loc=S_A \wedge \neg(Loc=S_B)$~~

~~(15) $\neg(A<10) \wedge \neg(A>5) \wedge \neg(Loc=S_A) \wedge Loc=S_B$~~

~~(16) $\neg(A<10) \wedge \neg(A>5) \wedge \neg(Loc=S_A) \wedge \neg(Loc=S_B)$~~

$A \geq 10$

Fragments finaux

F_2: $5 < A < 10 \wedge Loc = S_A$

F_3: $5 < A < 10 \wedge Loc = S_B$

F_6: $A \leq 5 \wedge Loc = S_A$

F_7: $A \leq 5 \wedge Loc = S_B$

F_{10}: $A \geq 10 \wedge Loc = S_A$

F_{11}: $A \geq 10 \wedge Loc = S_B$

Prise en considération de la sémantique

- Élimination des fragments contradictoires dépend de la sémantique des applications
- Exemple:
  - Si LOC est $\neq$ SA, $\neq$ SB, alors on ajoute les fragments suivants:

F4:	$5 < A < 10$	$\wedge$	$Loc \neq SA \wedge Loc \neq SB$
F8:	$A \leq 5$	$\wedge$	$Loc \neq SA \wedge Loc \neq SB$
F12:	$A \geq 10$	$\wedge$	$Loc \neq SA \wedge Loc \neq SB$

Fragmentation horizontale dérivée

- Soit S une table fragmentée horizontalement en w fragments
- Supposons qu'il y a un lien (clé étrangère) entre S et R
- Un fragment horizontal dérivé R_i de R est défini comme:

$$R_i = R \ltimes S_i \text{ , } 1 \leq i \leq w$$

avec:

S_i : un fragment horizontal de S

$\ltimes$: opération de semi-jointure

↓ algorithme facile à implémenter.

Problème de la fragmentation dérivée

- Le fait que le schéma d'une base de données est totalement connecté, on risque d'avoir plusieurs possibilités de fragmentation dérivée [Bellatreche, Entity-Relationship'98]
- Lorsqu'une relation a plusieurs parents (relation père/fils)

Comment choisir?

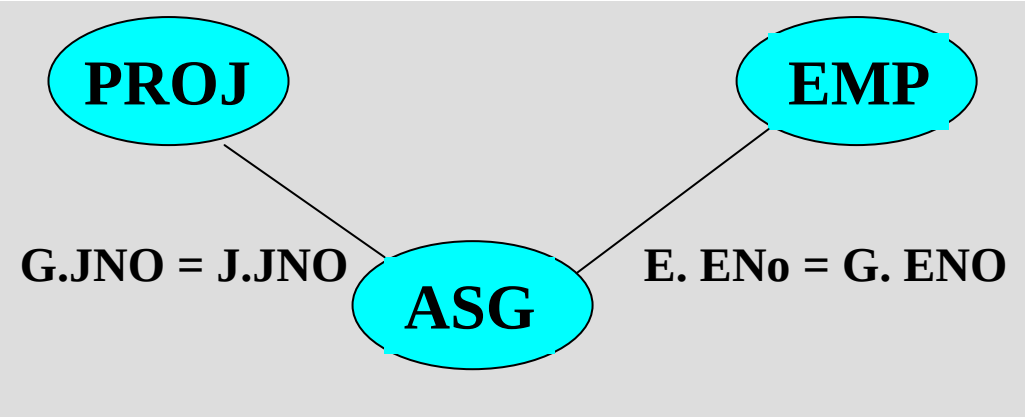
- Les caractéristiques de la jointure:
  - Choisir la solution qui optimise les opérations de jointure
- La performance d'une opération de jointure dans le contexte répartie dépend de la taille de fragments (ou relations) manipulés.

Jointure distribuée

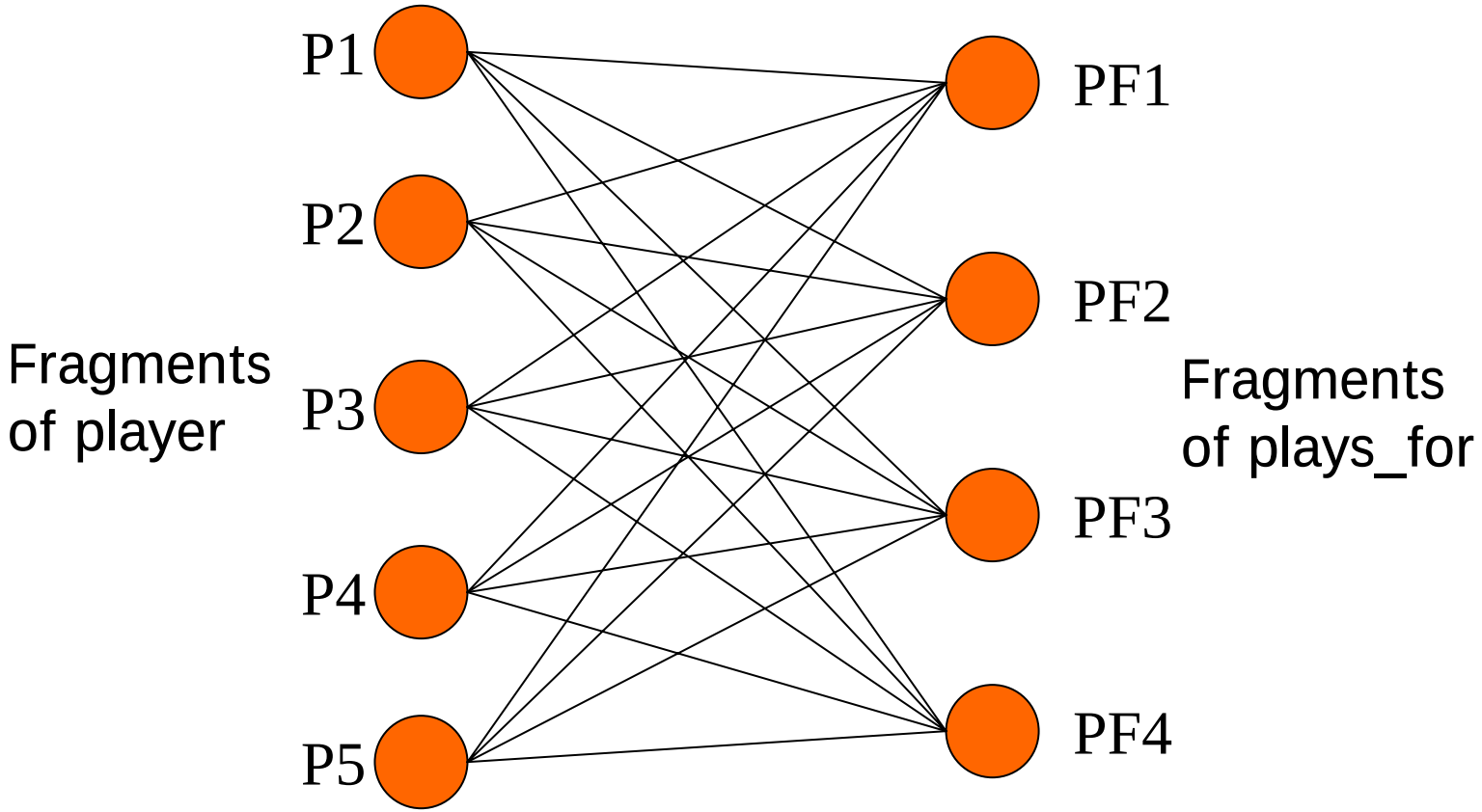
- Une jointure répartie est une jointure entre des relations horizontalement fragmentées
- Une jointure répartie peut être représentée par un graphe appelé graphe de jointure:
  - Les nœuds representent les fragments des tables partitionnées,
  - Une arête est introduite lorsque la jointure entre les fragments correspondants n'est pas vide

Graphe de jointure

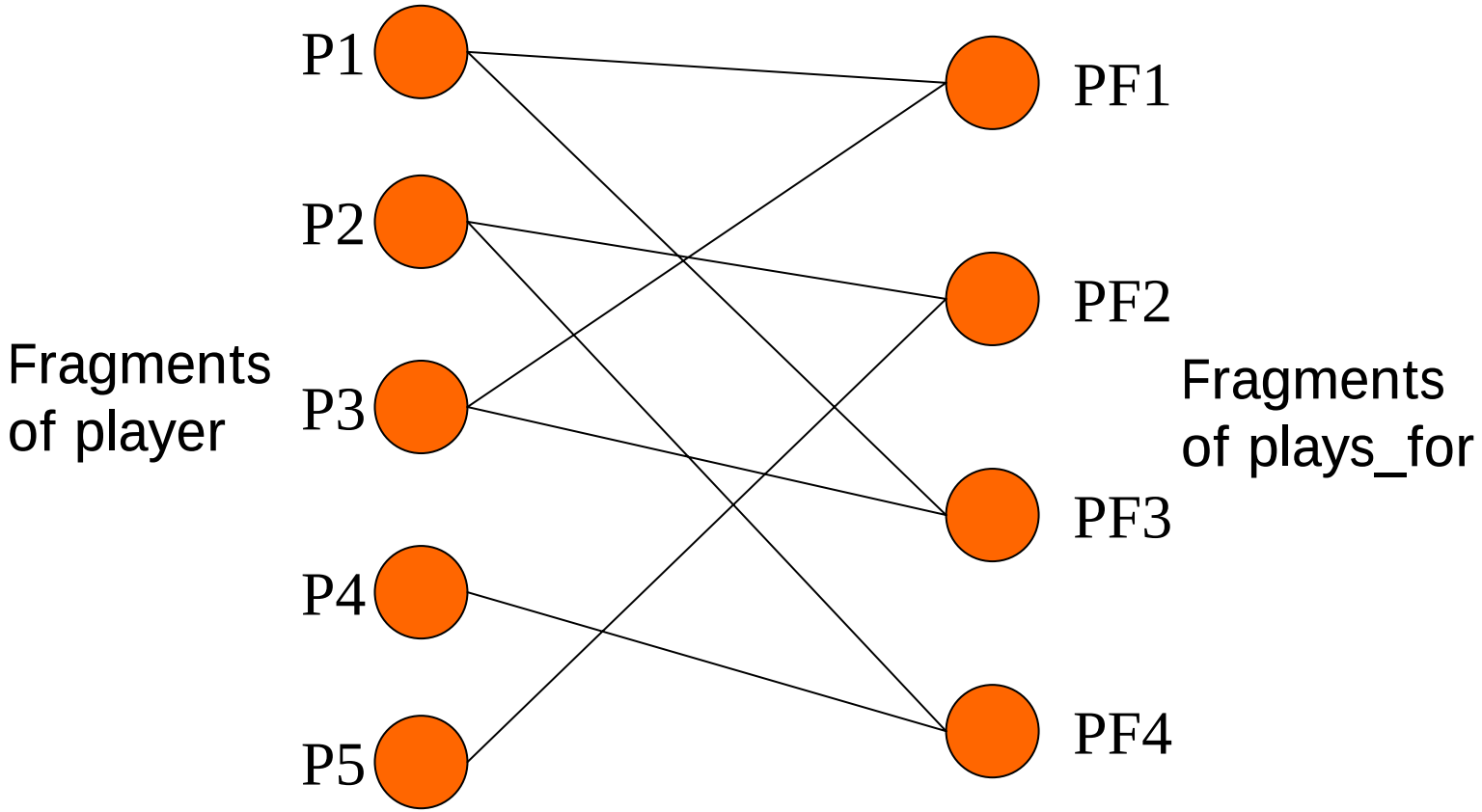
```
SELECT Ename, Resp
FROM E, G, J
WHERE E. ENo = G. ENO
AND G.JNO = J.JNO
AND JNAME = ``CAD''
AND DUR >= 36
AND Title = ``Prog''
```



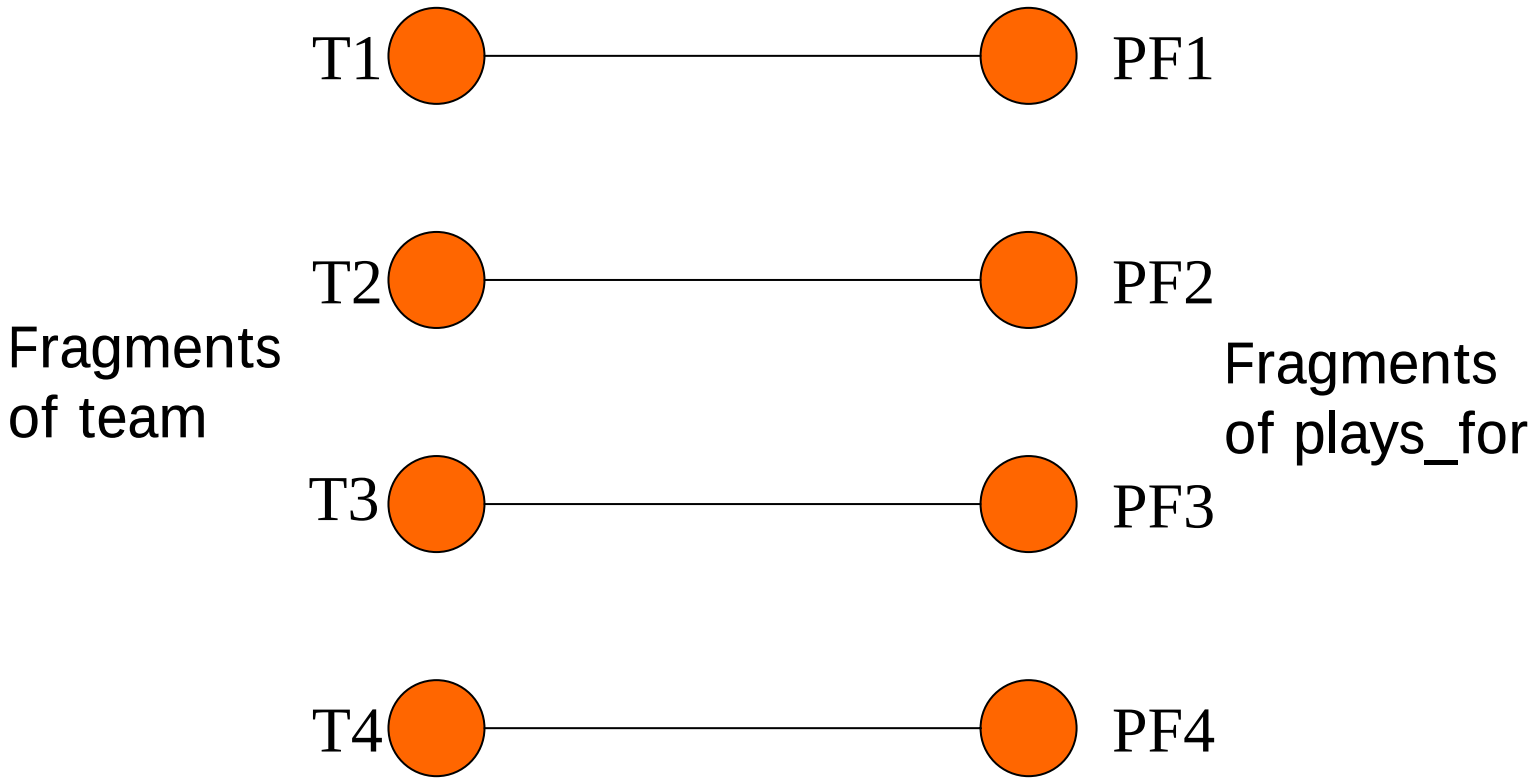
Graphe de jointure total



Graphe de jointure partiel



Graphe de jointure simple



Fragmentation verticale

Vertical Fragmentation: Algorithmes

- Pour une relation avec *m attributs (*non-primary key), le nombre de fragments verticaux possible est approximativement égale à m^m
- Techniques de fragmentation
  - **Groupement**: **(1)** chaque attribut forme un fragment, **(2)** jointure des fragments **tant qu'un critère de performance est satisfait.**
  - **Splitting**: on débute par la relation globale, on génère ensuite des partitions en se basant sur le comportement des applications (requêtes).

VF - Information: Besoins

- Principe:
  - Un fragment vertical contient des attributs fréquemment utilisés
  - Therefore, there is a need to have some measure of ``togetherness'' this is the measure of affinity among the attributes
- $Q = \{q_1, q_2, ..., q_m\}$: ensemble de requêtes fréquentes
- $R(A_1, A_2, ..., A_n)$: relation R avec n attributs, et l sites
- Matrice d'usage $m \times n$: U_{ij} : 1 si attribut A_j est référencé par q_i sinon 0
- Matrice d'accès $m \times l$: acc_{ik} : fréquence d'accès de la requête i sur le site k
- Matrice de référence m× l : ref_{ik} : le nombre d'accès aux attributs pour chaque exécution de la requête q_i sur le site k

Vertical Fragmentation - Exemple

- Relation PROJ(PNO,PNAME,BUDGET,LOC), 3 sites, 4 requêtes SQL :

  q_1: SELECT BUDGET FROM PROJ WHERE PNO = val;

  q_2: SELECT PNAME, BUDGET FROM PROJ;

  q_3: SELECT PNAME FROM PROJ WHERE LOC = val;

  q_4: SELECT SUM(BUDGET) FROM PROJ WHERE LOC=val;

$$U = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \qquad acc = \begin{bmatrix} 15 & 20 & 10 \\ 5 & 0 & 0 \\ 25 & 25 & 25 \\ 3 & 0 & 0 \end{bmatrix}$$

Mesure d'affinité

Mesure d'affinité entre deux attributs A_i et A_j (***aff_{ij}***) d'une relation R est définie comme:

$$\mathbf{aff_{ij} = \Sigma_{\{ k \mid U_{ki} = 1 \wedge U_{kj} = 1\}} \Sigma_l \, acc_{lk}}$$

$$U = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \quad acc = \begin{bmatrix} 15 & 20 & 10 \\ 5 & 0 & 0 \\ 25 & 25 & 25 \\ 3 & 0 & 0 \end{bmatrix} \quad aff = \begin{bmatrix} 45 & 0 & 45 & 0 \\ 0 & 80 & 5 & 75 \\ 45 & 5 & 53 & 3 \\ 0 & 75 & 3 & 78 \end{bmatrix}$$

Matrice AA

Matrice d'affinité et son rôle pour la FV

- La matrice d'affinité est utilisée pour guider la génération des fragments verticaux
- les attributs ayant une grande affinité seront groupés pour former un fragment vertical
- Algorithmes de fragmentation:
  - Algorithmes basés sur un graphe [Bellatreche & Karlapalem: Distributed and parallel database journal-2000]

Principe de l'algorithme

- Construction d'un graphe d'affinité des attributs (graphe complet)
  - Nœuds : les attributs
  - Arête entre A_i et A_j est associée à la valeur d'affinité entre A_i et A_j
- Génération des cycles ayant une grande affinité
- Chaque cycle représente un fragment vertical

Processus d’allocation des fragments

Allocation des fragments

- Problème d'allocation:
  - Entrées:
    - $F = \{F_1, F_2, ..., F_n\}$ - ensemble de fragments
    - $S = \{S_1, S_2, ..., S_m\}$ - ensemble de sites
    - $Q = \{Q_1, Q_2, ..., Q_l\}$ - ensemble de requêtes
  - Le problème d'allocation consiste à trouver une distribution optimale de F sur S afin d'améliorer la performance (temps de réponse, données transférées, etc.)
- Contraintes
  - Stockage, équilibrage de la charge entre les sites

➔ Problème NP-complet

Alternatives d'allocation

- Réplication totale
  - chaque fragment est répliqué sur tous les sites
- Réplication partielle
  - chaque fragment est répliqué sur quelques sites
- Aucune réplication
  - Chaque fragment réside dans un et un seule site

Duplication

- Favoriser les accès locaux
- Augmenter la disponibilité des données
  - Stockage
  - Gestion des mises à jour
    - Utilisation des triggers pour détecter des mises à jour

Trigger

- Action sur la base à exécuter suite à l'apparition d'un événement particulier
- ECA: Evénement | Condition | Action
- Exemple

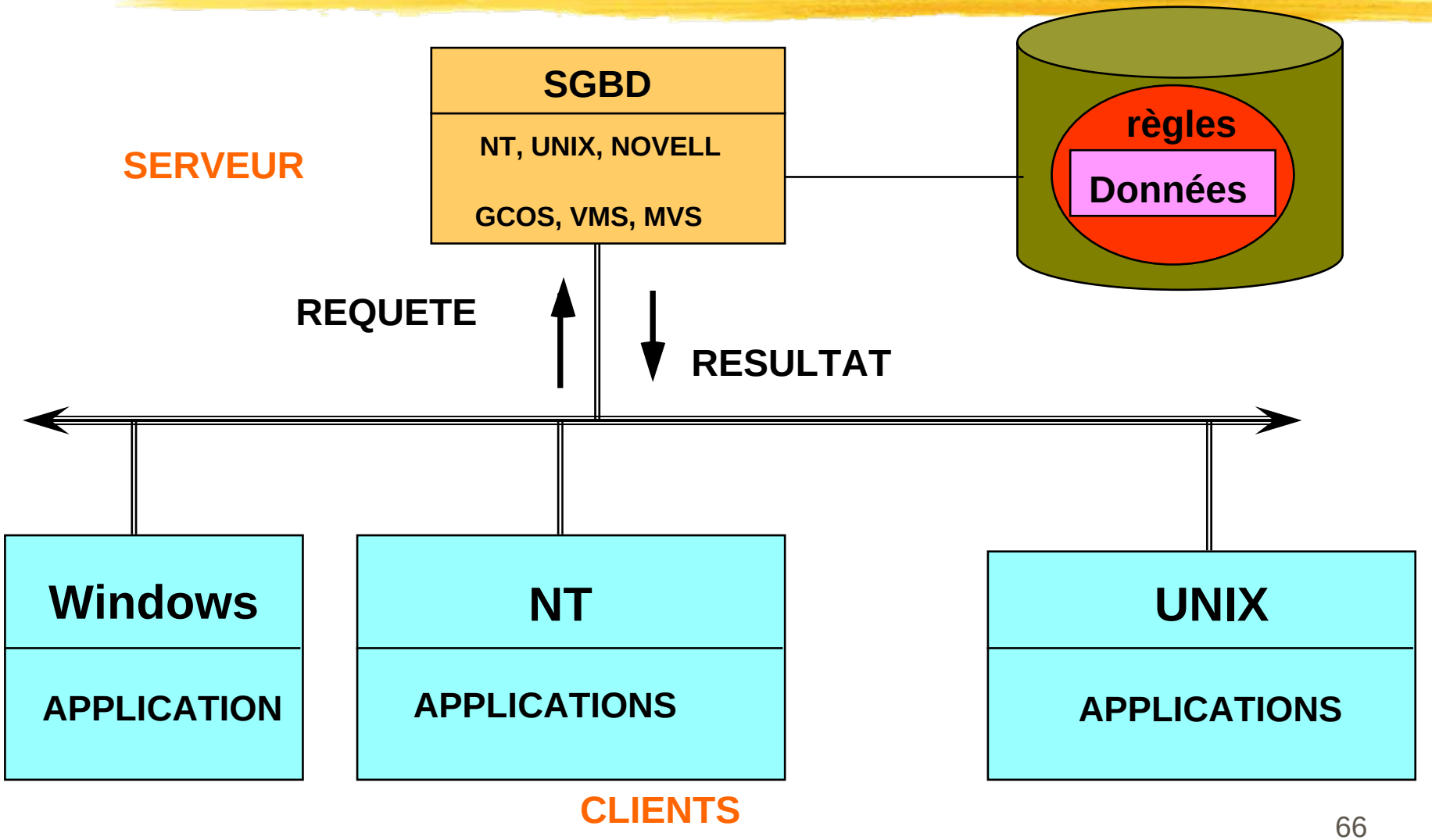
```
Employé(ID int, Salaire float)
Cumul(ID int, Augmentation float)
create trigger
after update of salaire on Employé
referencing old as ancien_salaire, new as nouveau_salaire
update Cumul
set Augmentation = Augmentation + nouveau_salaire - ancien_salaire
where ID = Employé.ID
```

Architectures

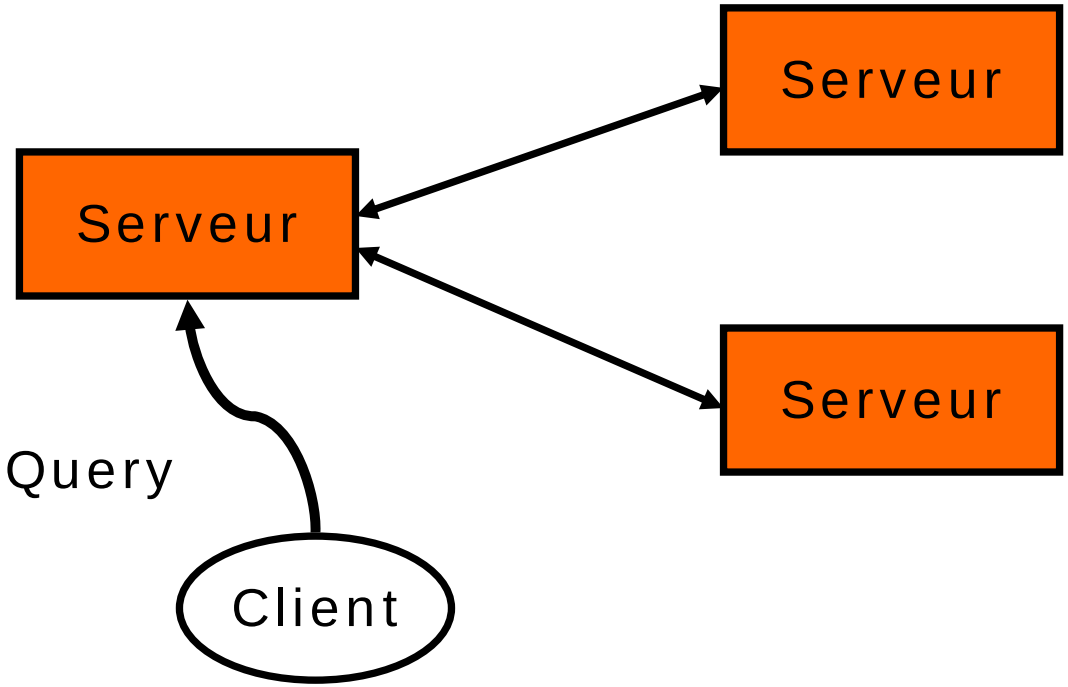
Client/Serveur

- Modèle d'architecture applicative où les programmes sont répartis entre processus clients et serveurs communiquant par des requêtes avec réponses.
- Une répartition hiérarchique des fonctions
  - Données sur le serveur partagées entre N clients
  - interfaces graphiques sur la station de travail personnelle
  - Communication par des protocoles standardisés
  - Distribution des programmes applicatifs afin de minimiser les coûts

Architecture C/S

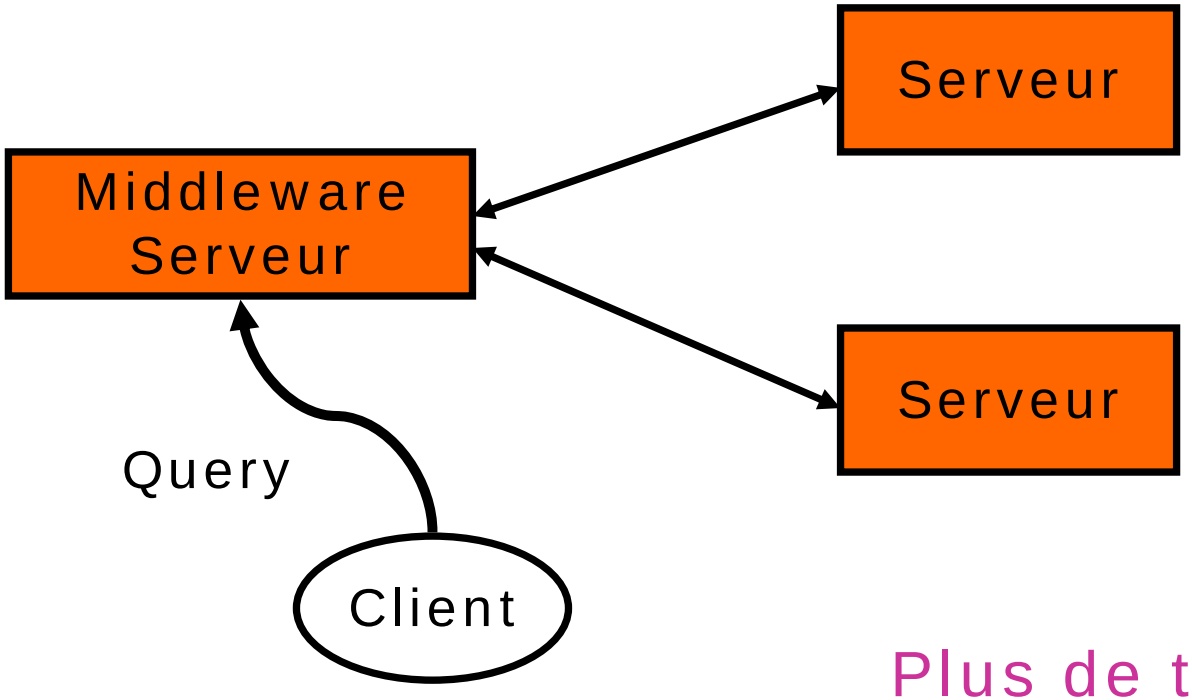


Collaborating Servers



Système de Middleware

Un serveur (middleware) gère les requêtes et les transactions



Plus de transparence

Peer-to-peer (P2P)

- Aucune différence entre le client et le serveur
- Le client peut jouer le rôle de serveur et vice versa
- Chaque machine a sa propre base de données (+SGBD)
- Les BDR dans le contexte P2P reste un sujet de recherche!!

Optimisation de Requêtes

Contexte Réparti

Évaluation des requêtes dans un environnement réparti

- Introduction
- Évaluation des requêtes réparties
  - Décomposition des requêtes
  - Localisation des données
- Optimisation des requêtes

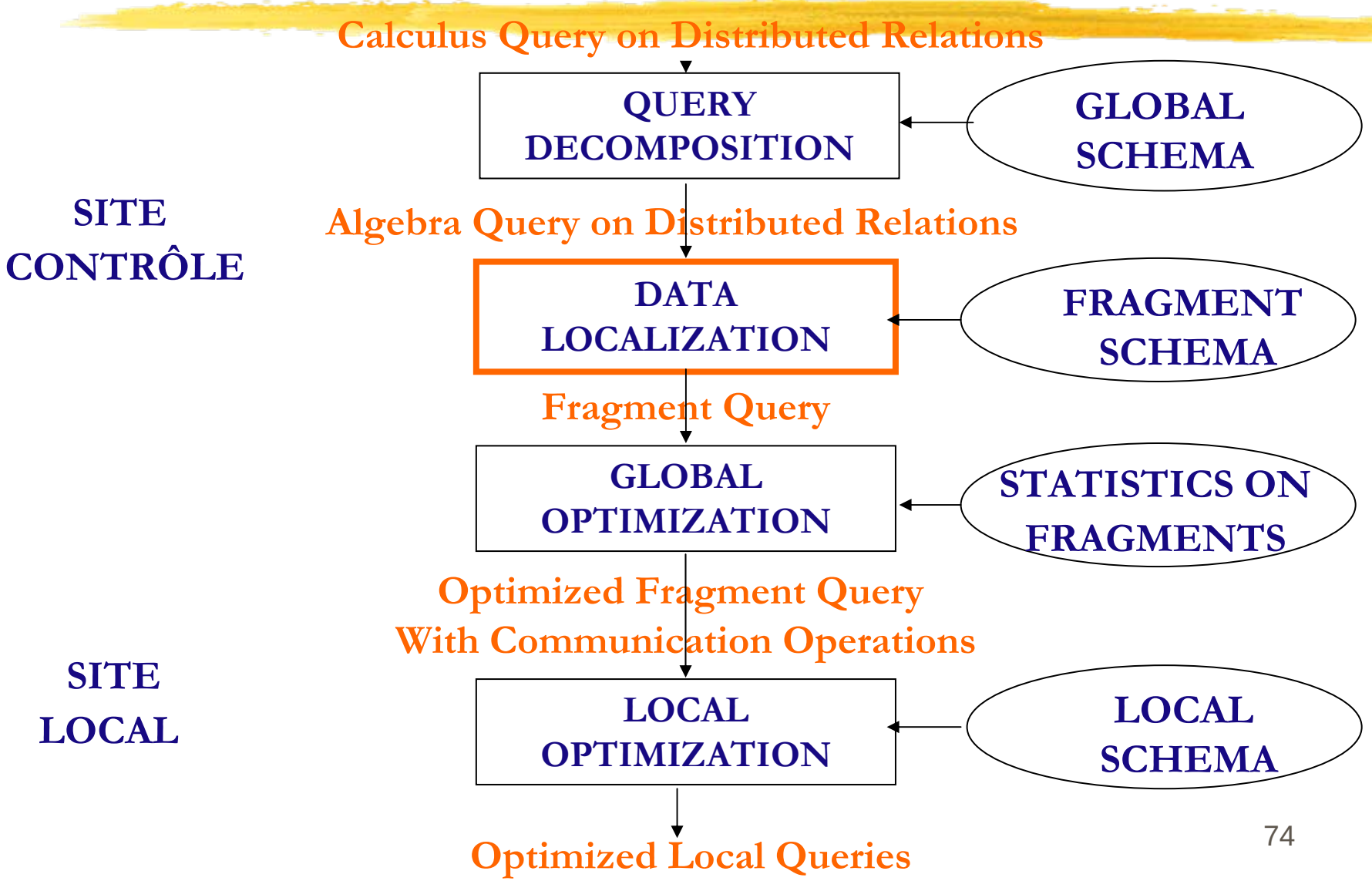
Introduction

- Optimisation des requêtes dans les BDRs
  - Plus complexe que dans les BDs centralisées
  - Choisir les sites où chercher les données et effectuer les traitements
  - Minimiser le coût = E/S + UCT + *Com*
    - La pondération de chaque composante dépend du type de réseau (exemple: communication dominante dans WAN)

Mesures de performances

- Nombre d'accès en mémoire secondaire (I/O)
- Temps CPU
- Délai de réponse ("elapsed time")
- Espace en mémoire secondaire
- Débit des transactions ("throughput")
- Temps de communication (BD répartie)

Couches d'un optimiseur de requêtes réparti



Caractéristiques de l'optimiseur réparti (I)

- Statistiques
  - Cardinalité des fragments
  - Taille d'un tuple
  - Valeurs distinctes des tuples
  - etc.
- Sites de décision
  - Un ou plusieurs sites participant dans la sélection des stratégies
- Exploitation de la topologie du réseau
  - Wide area network
  - Local area network

Caractéristiques de l'optimiseur réparti (II)

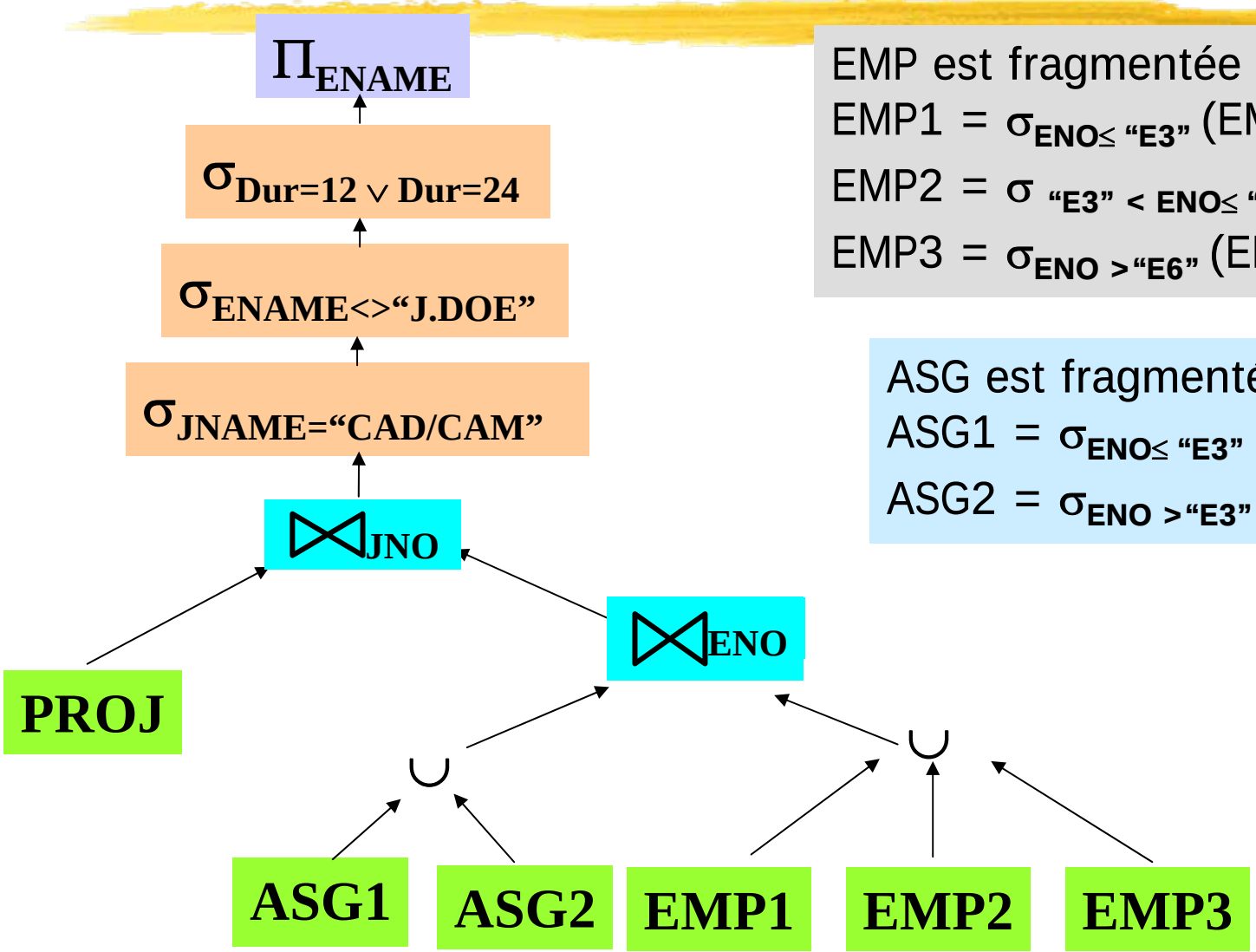
- Exploitation des fragments répliqués
  - un nombre important de stratégies
- Utilisation des opérations de semi-jointures
- Réduction de la taille des données transférées
- Augmentation des traitements locaux

Localisation des données

Input: requête algébrique

- Détermination des fragments concernés par cette requête
- Localisation des fragments
- Exécution de la requête sur les fragments

Exemple



EMP est fragmentée en:

EMP1 = $\sigma_{ENO \leq \text{“E3”}}$ (EMP)

EMP2 = $\sigma_{\text{“E3”} < ENO \leq \text{“E6”}}$ (EMP)

EMP3 = $\sigma_{ENO > \text{“E6”}}$ (EMP)

ASG est fragmentée en:

ASG1 = $\sigma_{ENO \leq \text{“E3”}}$ (ASG)

ASG2 = $\sigma_{ENO > \text{“E3”}}$ (ASG)

Réduction avec la Sélection

SELECT *
FROM EMP
WHERE ENO="E5"

EMP est fragmentée en:

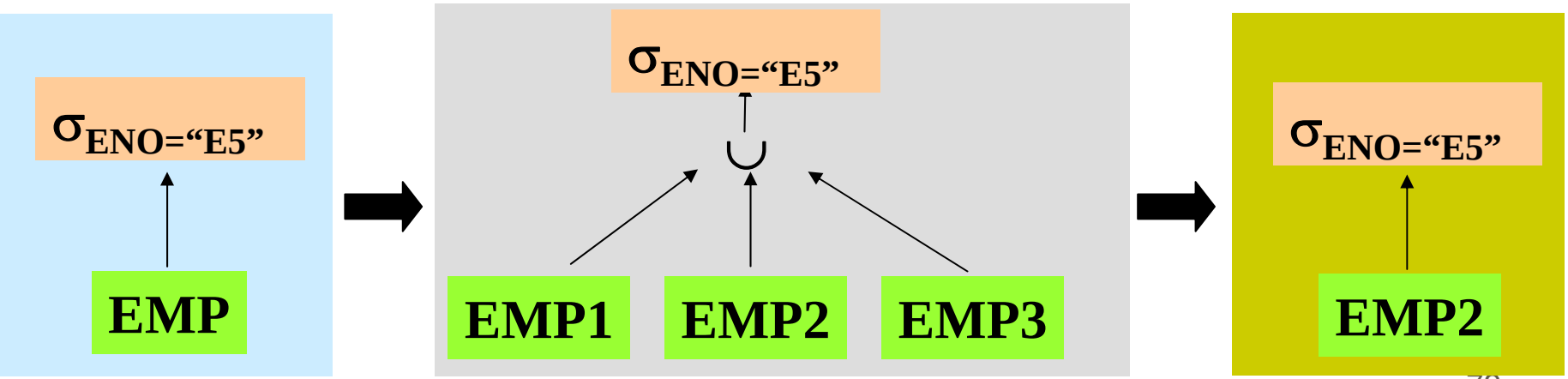
EMP1 = $\sigma_{ENO \leq \text{"E3"}}$ (EMP)

EMP2 = $\sigma_{\text{"E3"} < ENO \leq \text{"E6"}}$ (EMP)

EMP3 = $\sigma_{ENO > \text{"E6"}}$ (EMP)

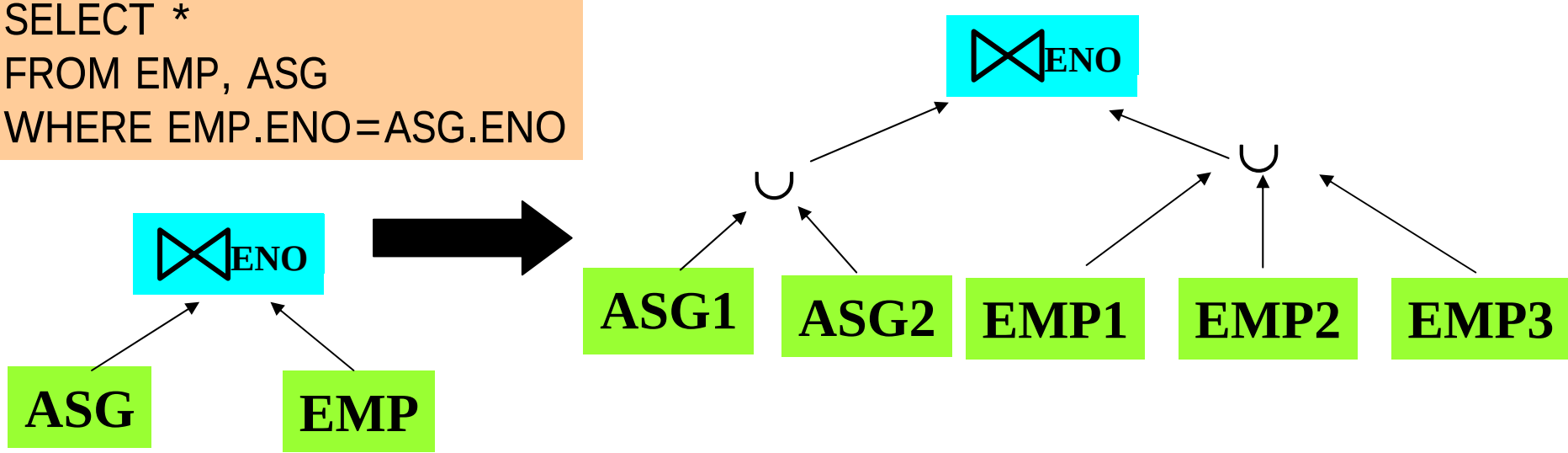
Soit une relation R, $F_R=\{R_1, R_2, \ldots, R_n\}$ avec $R_j = \sigma_{p_j}(R)$

$\sigma_{pi}(R_j) = \varnothing$ si $\forall x \in R: \neg(p_i(x) \wedge p_j(x))$



Réduction avec jointure (I)

```
SELECT *
FROM EMP, ASG
WHERE EMP.ENO=ASG.ENO
```



ASG est fragmentée en:

ASG1 = $\sigma_{ENO \leq \text{"E3"}}$ (ASG)

ASG2 = $\sigma_{ENO > \text{"E3"}}$ (ASG)

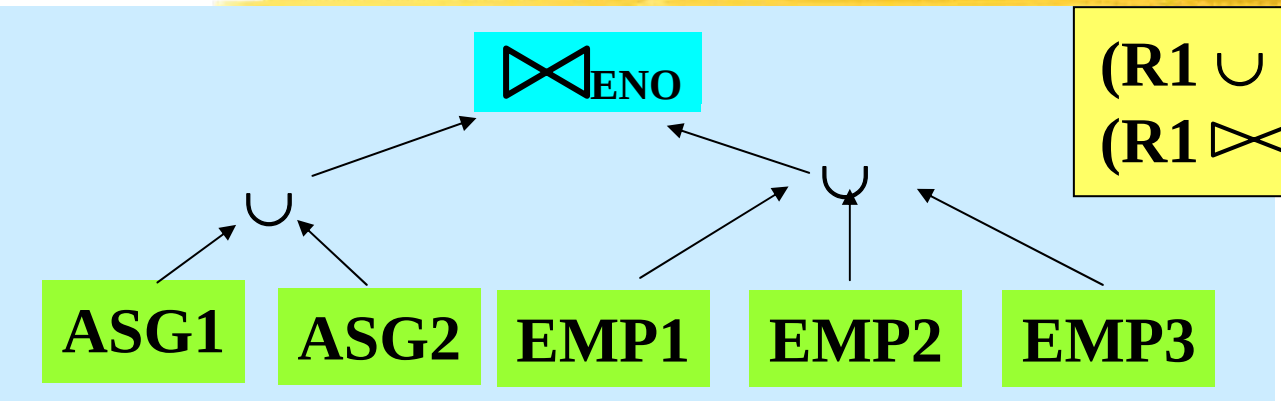
EMP est fragmentée en:

EMP1 = $\sigma_{ENO \leq \text{"E3"}}$ (EMP)

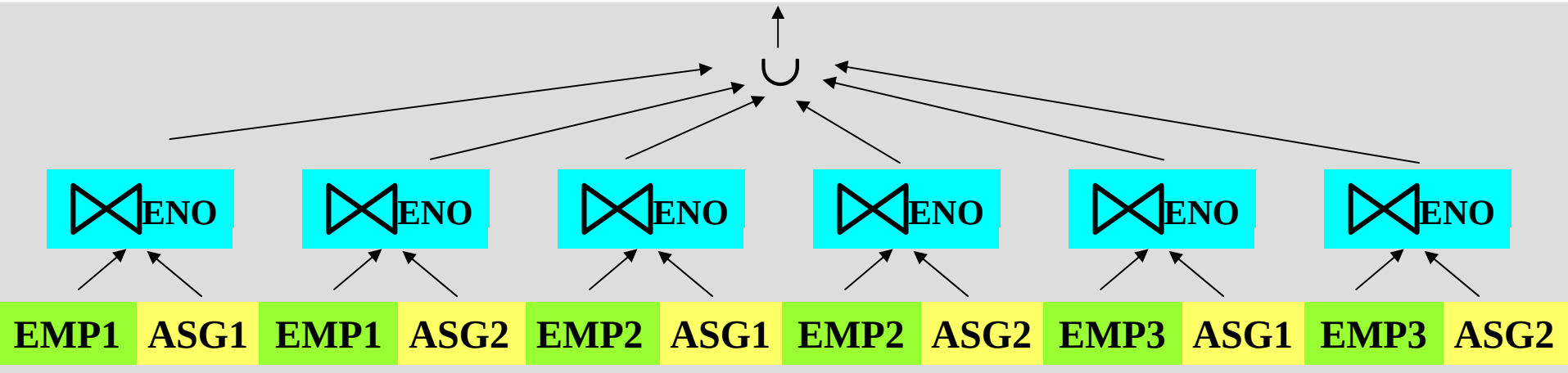
EMP2 = $\sigma_{\text{"E3"} < ENO \leq \text{"E6"}}$ (EMP)

EMP3 = $\sigma_{ENO > \text{"E6"}}$ (EMP)

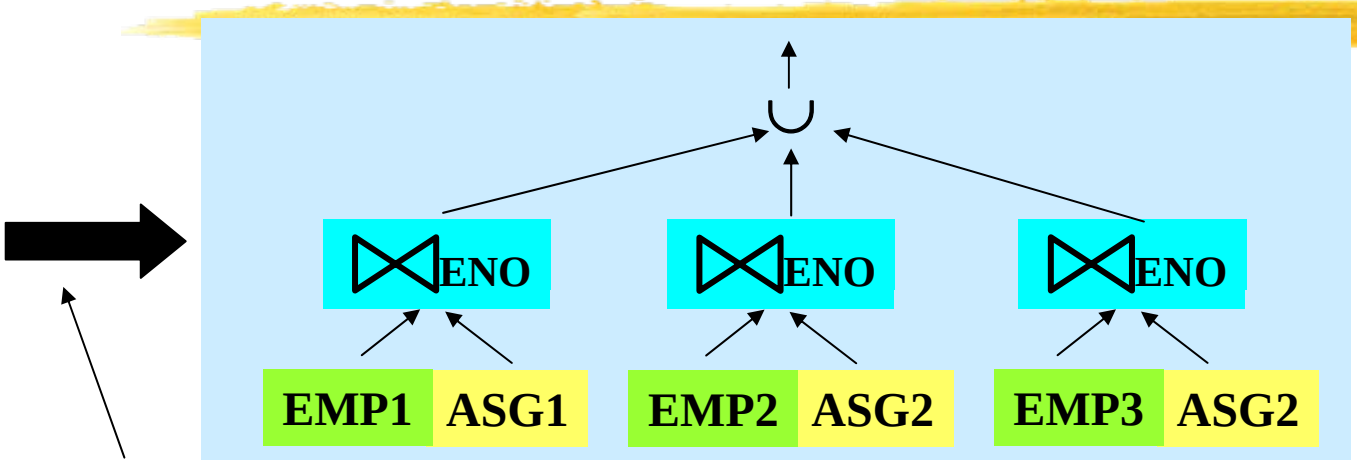
Réduction avec jointure (II)



$$(R1 \cup R2) \bowtie S \Leftrightarrow (R1 \bowtie S) \cup (R2 \bowtie S)$$



Réduction avec jointure (III)



Given $R_i = \sigma_{pi}(R)$ and $R_j = \sigma_{pj}(R)$

$R_i \bowtie Rj = \emptyset$ if $\forall x \in R_i$, $\forall y \in R_j$: $\neg(p_i(x) \wedge p_j(y))$

Réduction avec une jointure:

1. **Distribuer la jointure sur l'union**
2. **Éliminer les opérations non nécessaires**

Réduction pour la VF

- Trouver les relations intermédiaires non utilisées

  Relation R sur A = {A1, A2, ..., An} fragmentée verticalement, tel que chaque R_i est défini comme suit:

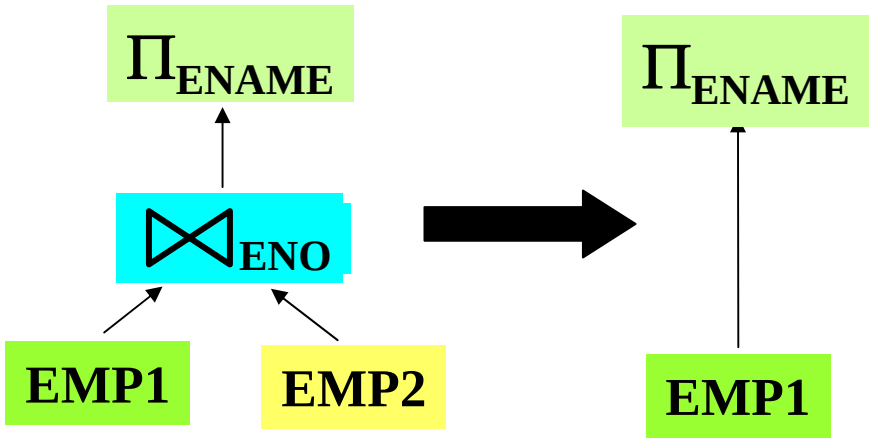
  $R_i = \Pi_{A'}(R)$ avec $A' \subseteq A$

  $\Pi_{K,D}(R_i)$ est non nécessaire si l'ensemble D de la projection n'est pas dans A'

EMP1= $\Pi_{ENO,ENAME}$ (EMP)

EMP2= $\Pi_{ENO,TITLE}$ (EMP)

```
SELECT ENAME
FROM   EMP
```



Réduction pour FHD (I)

Distribuer les jointures sur les unions

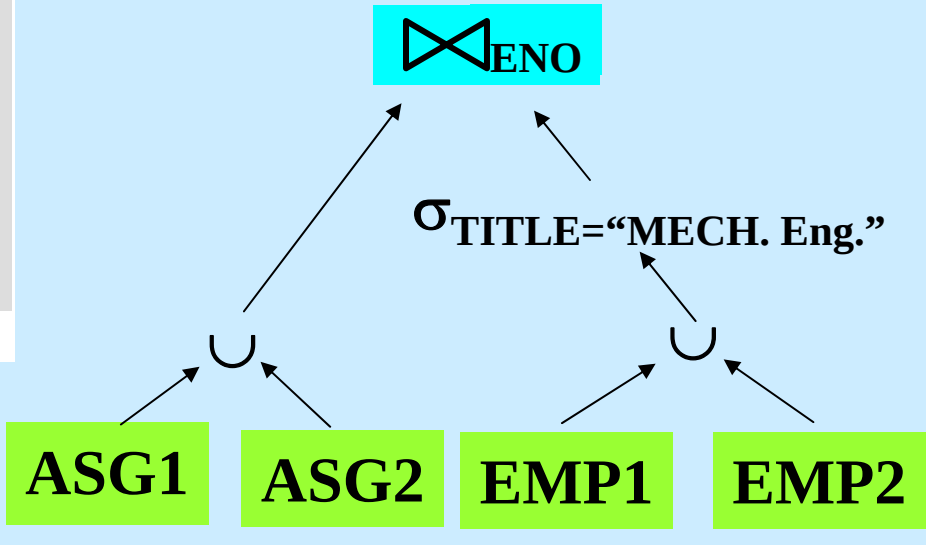
appliquer la réduction des jointures sur la FH

EMP1: $\sigma_{TITLE=\text{“Programmer”}}$ **(EMP)**

EMP2: $\sigma_{TITLE\neq\text{“Programmer”}}$ **(EMP)**

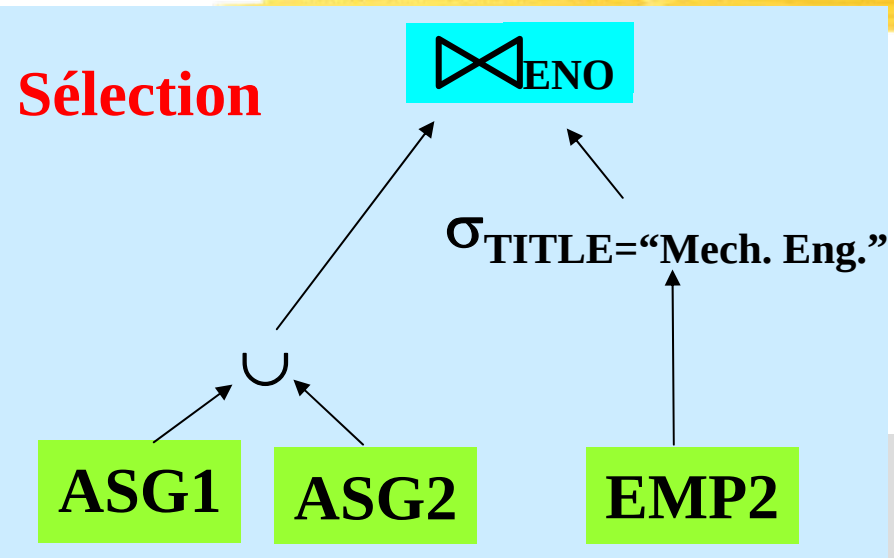
ASG1: ASG $\ltimes_{ENO}$ **EMP1**

ASG2: ASG $\ltimes_{ENO}$ **EMP2**

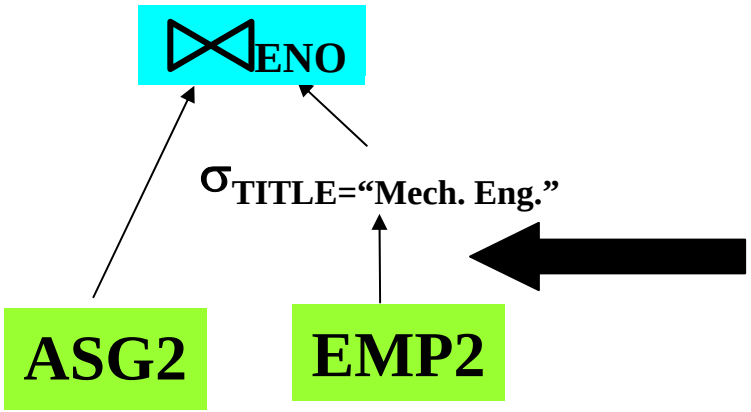


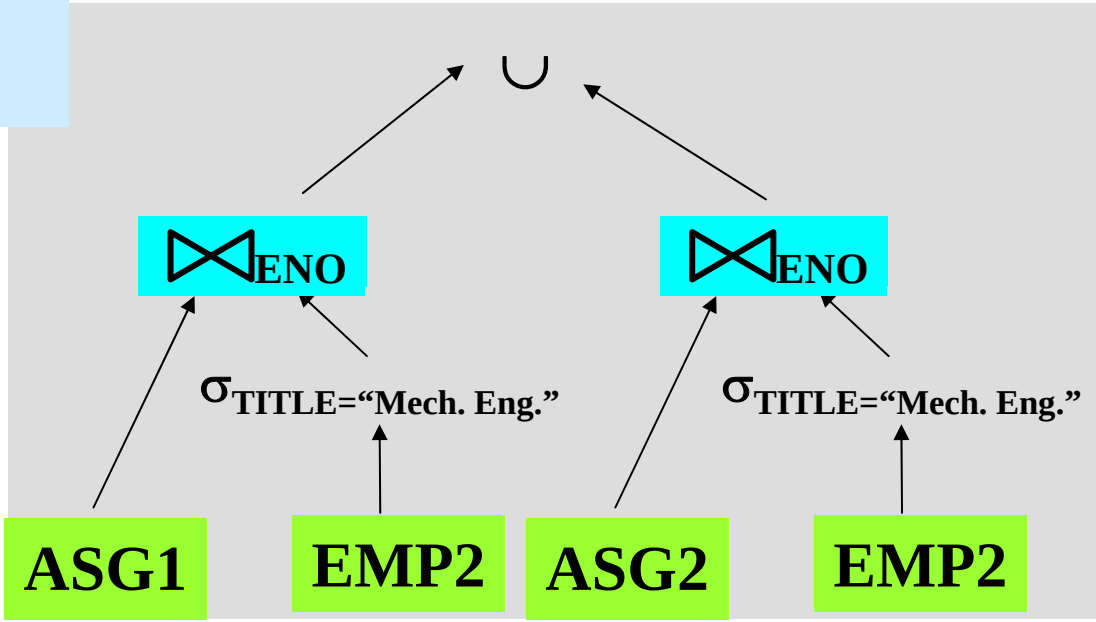

```
SELECT *
FROM EMP, ASG
WHERE ASG.ENO = EMP.ENO
AND EMP.TITLE = “Mech. Eng.”
```

Réduction pour FHD (II)



Jointure sur union





Réduction pour la FH

- Éliminer les relations vides générées par les sélections contradictoires sur les fragments horizontaux;
- Éliminer les relations non nécessaires générées par la projection sur les fragments verticaux;
- Distribuer les jointures sur les unions

Exemple

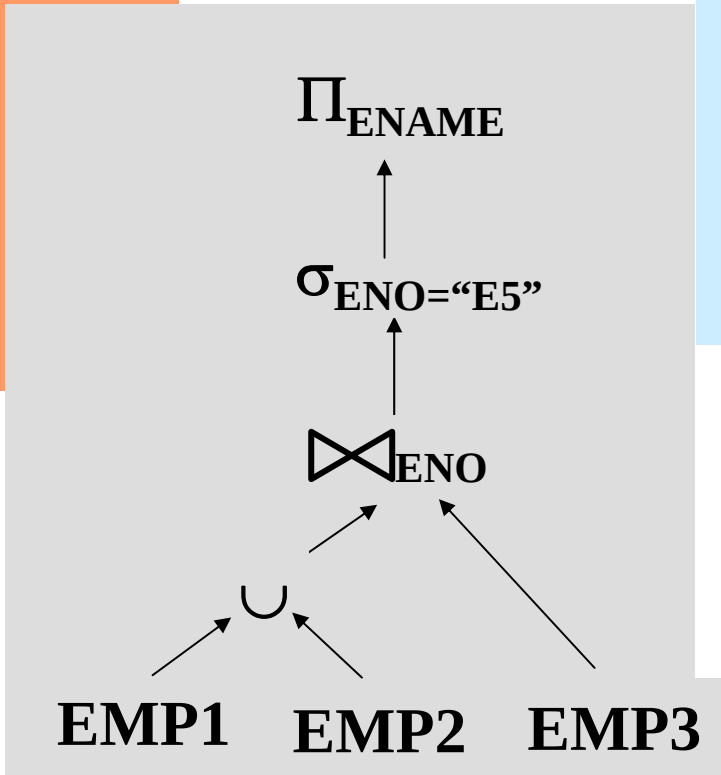
EMP1 = $\sigma_{ENO \leq \text{“E4”}} (\Pi_{ENO,ENAME} (EMP))$

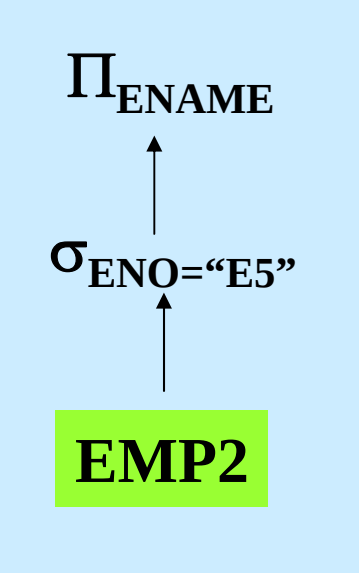
EMP2 = $\sigma_{ENO > \text{“E4”}} (\Pi_{ENO,ENAME} (EMP))$

EMP3 = $\Pi_{ENO,TITLE} (EMP)$

Requête

```
SELECT ENAME
FROM EMP
WHERE ENO = "E5"
```





Pourquoi optimiser? exemple

Base de données:

EMP(eno, ename, title)

ASG(eno, jno, resp, dur)

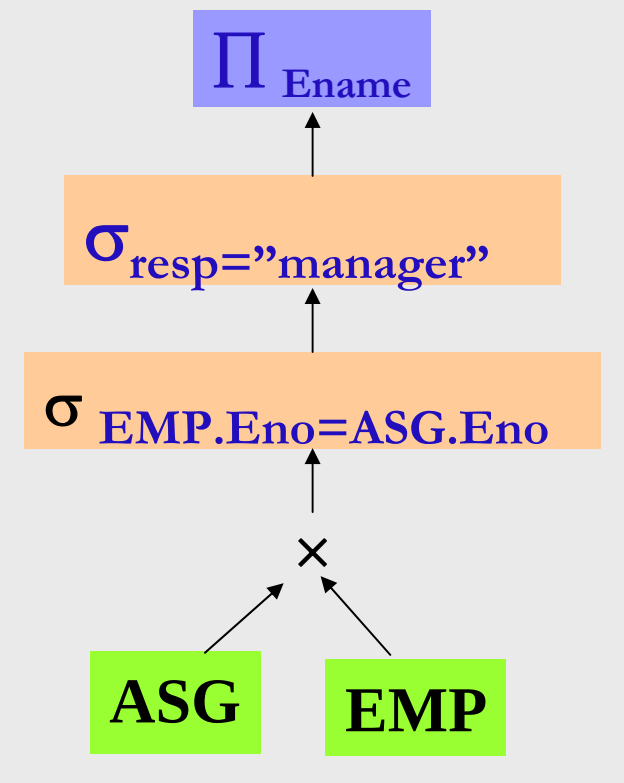
Requête

Lister les noms des employés gérant un projet

SQL

```
Select ename
From EMP e, ASG g
Where e.Eno = g. Eno
And resp = "manager"
```

Arbre algébrique



Exemple - Stratégies

Schéma des fragments

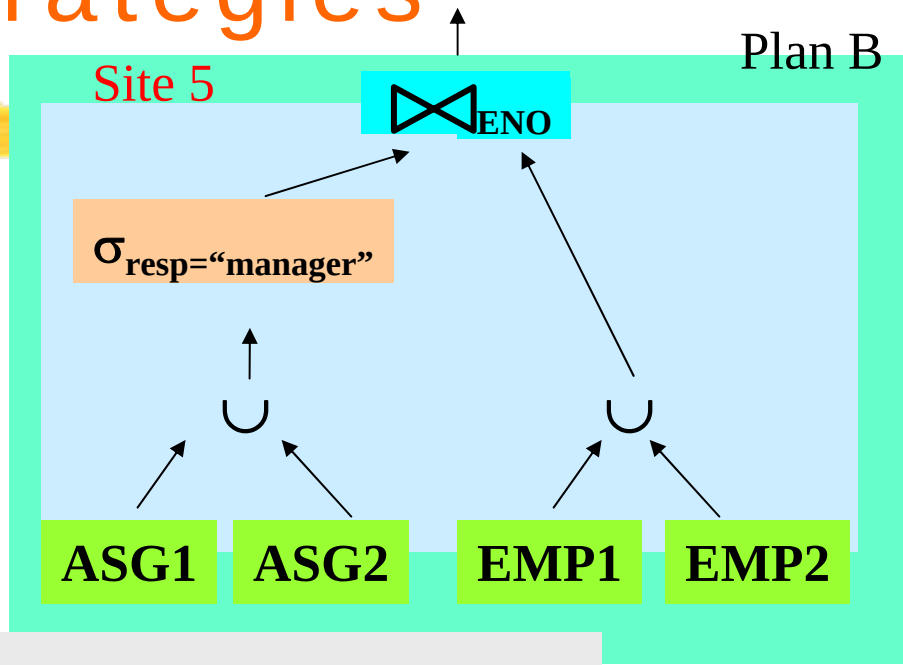
$EMP1 = \sigma_{ENO <= 100}(EMP)$ at site 1

$EMP2 = \sigma_{ENO > 100}(EMP)$ at site 2

$ASG1 = \sigma_{ENO <= 100}(ASG)$ at site 3

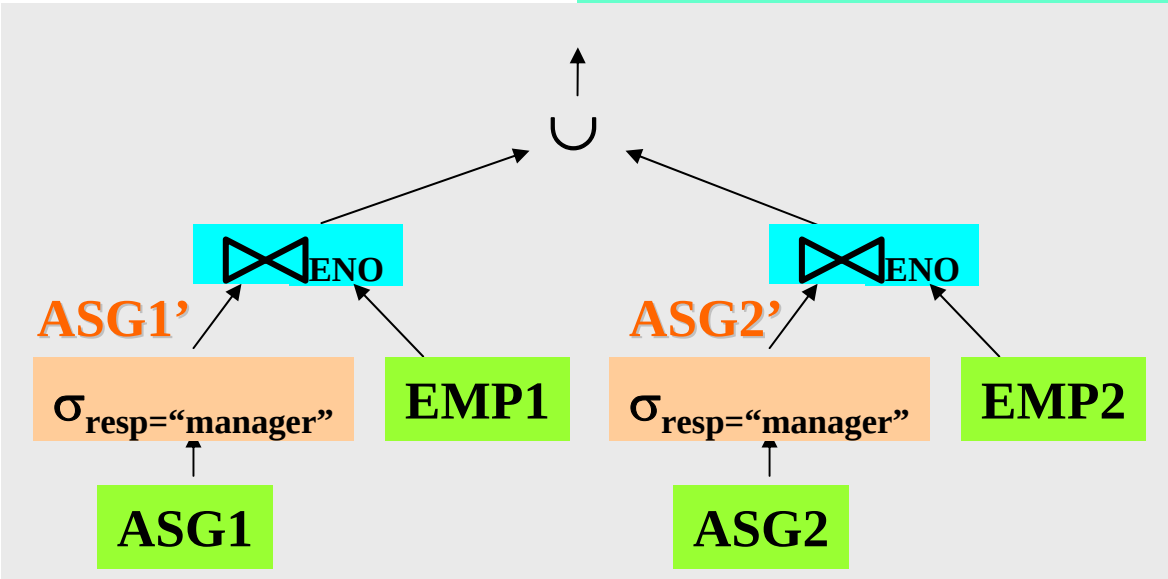
$ASG2 = \sigma_{ENO > 100}(ASG)$ at site 4

Plan B



Site de requête:
Site 5

Plan A



Statistiques & coûts

Statistiques sur la base:

- ||EMP|| = 400 tuples,
- ||ASG|| = 1000 tuples,
- 20 managers dans ASG
- Les données sont uniformément distribuées sur les sites

Coûts:

- Accès tuple: t_{acc} = 1 unit
- Transfert d'un tuple: t_{trans} = 10 units

Coûts des plans

⌘ Coût du Plan A:

- Produire ASG' = 20 $*$ t_{acc} = 20 (traitement local)
- Transférer de ASG' = 20 *t_{trans} = 200 (transfert sur le site de EMP)
- Produire EMP' = (20 * 400) * t_{acc} = 8000 (jointure sur le site EMP)
- Transférer EMP' = 20 * t_{trans} = 200 (envoie sur le Site 5)

Coût Total = 8420

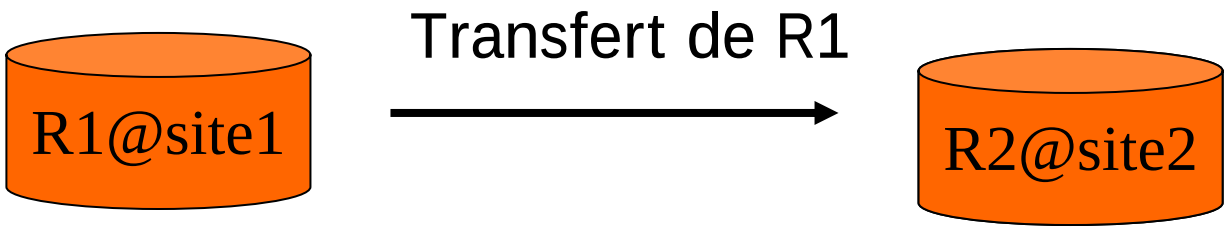
⌘ Coût du Plan B:

- Transférer EMP = 400 * t_{trans} = 4,000 (sur le Site 5)
- Transférer ASG = 1000 * t_{trans} = 10,000 sur le site Site 5)
- Produire ASG' = 1000 * t_{acc} = 1,000 (sélection sur Site 5)
- Jointure de EMP et ASG' = 400 * 20 * t_{acc} = 8,000 (sur let Site 5)

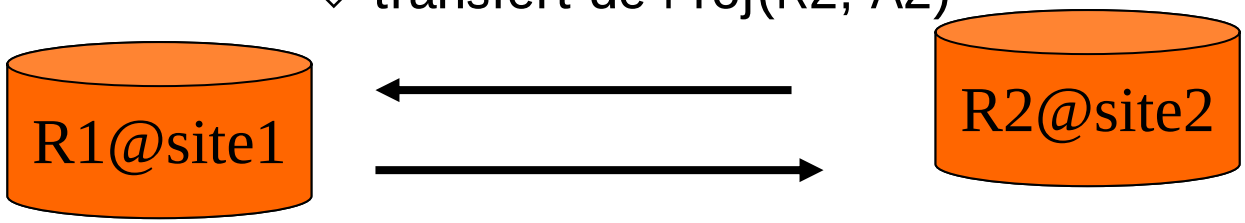
Total cost = 23,000

Optimisation : jointure répartie

Jointure (R1, R2, R1.A1 = R2.A2) @ site 2



- transfert de Proj(R2, A2)



- transfert de Proj(Join(R1, Proj(R2,A2), R1.A1 = R2.A2), R1.*)

System R* (IBM)

Système R*: optimisation de requêtes réparties

- Coût total: inclus le coût local des traitements et le coût de transmission
- Algorithme
  - Pour chaque relation dans l'arbre algébrique, trouver le meilleur plan d'exécution (voir le cours sur l'optimisation de requêtes)
  - Pour la jointure de n relation, trouver le meilleur ordre
  - Chaque site local optimise ses traitements

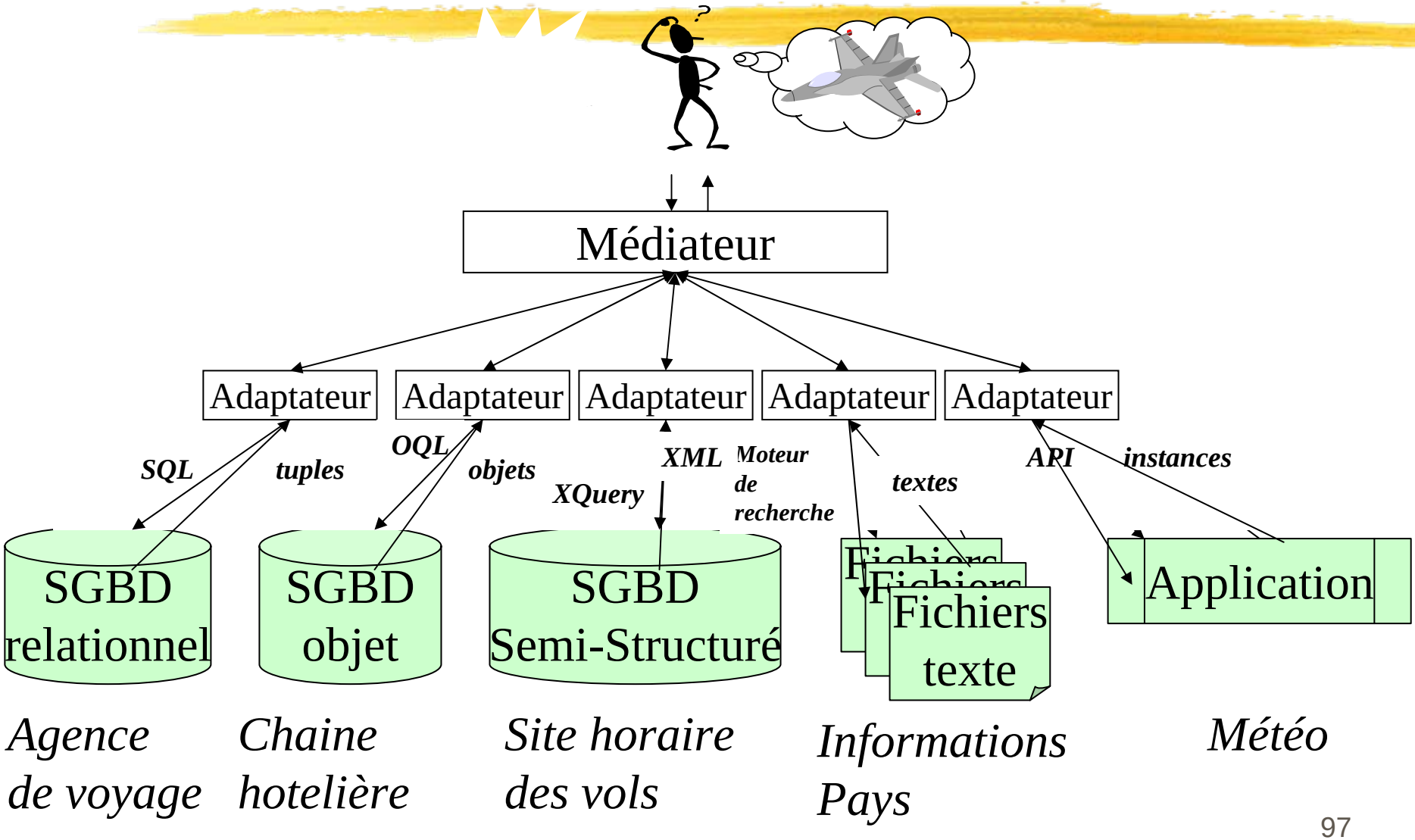
Stratégies de transfert de données

- Ship-whole: une relation (de jointure) entière est transférée et stockée temporairement, ensuite on utilise l'algorithme de jointure de type *sort merge*

- Fetch-as-needed: cette stratégie est équivalente à la semi jointure

Bases de données hétérogènes

Architecture de médiation



Adaptateur (Wrapper)

- L'adaptateur (Wrapper) s'occupe de l'hétérogénéité des sources. C'est un "traducteur".
  - Traduction du langage de requête commun en langage de requête natif (propre à la source)
  - Traduction des résultats natifs en résultats au format commun

Médiateur

- Le médiateur s'occupe de la distribution des sources
  - Localisation des sources
  - Décomposition des requêtes en requête adaptée pour chacune des sources
  - Envoi des requêtes aux sources
  - Recomposition des différents résultats provenant de chacune des sources

Architecture GAV et LAV

- **GAV : Global as View**
  - Schéma global défini comme une vue intégrante sur schémas locaux
  - Approche ascendante depuis les sources vers le médiateur
  - Difficulté pour ajouter une source
- **LAV : Local As View**
  - Chaque source locale est définie comme une vue locale du schéma global
  - Approche descendante depuis le médiateur vers les sources
  - Difficulté pour réconcilier source et vue locale